

ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет вычислительной техники

Кафедра «Математическое обеспечение и применение ЭВМ»

Курсовая работа

по дисциплине

«Базы и хранилища данных»

на тему: «Программные средства ведения данных библиотекаря в единой базе
данных информационного сервиса сети городских библиотек»

ПГУ 090304.005

Направление подготовки – 09.03.04

Программная инженерия

Выполнил студент: Гришин Гришин М.М.

Группа: 22ВП2

Руководитель работы: Шибанов Шибанов С.В.

к.т.н., доцент

Работа защищена с оценкой 057

Преподаватели _____

Дата защиты 17.12.25

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ

Пензенский государственный университет

«УТВЕРЖДАЮ»

Зав. кафедрой МОиПЭВМ.

А. Ю. Козлов

ЗАДАНИЕ

на курсовую работу по дисциплине
«Базы и хранилища данных»

Студенту _____ Гришину М. М. _____ Группа 22ВП2

Тема: «Программные средства ведения данных библиотекаря в единой базе данных информационного сервиса сети городских библиотек»

Исходные данные (технические требования) на проектирование

1. Провести первичный анализ предметной области (ПрОбл), выделить категории пользователей информационной системы (ИС).
2. Определить автоматизируемые бизнес-процессы для заданной категории пользователей: не менее трех бизнес-процессов (БП) учетного характера (в т. ч., по крайней мере, один БП создания и добавления в БД коллекции информационных объектов в рамках единой транзакции; не менее одного БП, реализующего модификацию одного или нескольких информационных объектов в базе данных; не менее одного БП, реализующего явное или виртуальное удаление одного или нескольких информационных объектов в базе данных, не менее одного БП, выполняемого автоматически для при наступлении в базе данных определенных событий, связанного с модификацией данных и не менее двух БП, которые должны формировать агрегированные группы данных, актуальные для анализа информации и возможного принятия решений).
3. Провести моделирование автоматизируемых бизнес-процессов для заданной категории пользователей ИС, определить функциональные требования и специфицировать варианты использования пользовательского приложения ИС, оформить диаграмму вариантов использования UML.
4. Разработать архитектуру, определить состав и функциональные требования к разрабатываемым программным средствам ведения данных, которые должны выполнять операции с данными в контексте автоматизируемых бизнес-процессов и предоставлять программные интерфейсы другим компонентам пользовательского приложения.
5. На основе результатов анализа ПрОбл провести моделирование БД ИС с применением современных case-средств, оформить соответствующие диаграммы и отчеты по моделям БД, развернуть БД в среде целевой системы управления базами данных (СУБД).
6. Программное обеспечение разрабатываемого приложения должно быть полностью отлажено и протестировано, необходимо привести описание процесса и результаты тестирования.

1. Объем работы по курсовому проекту

1. Анализ предметной области и постановка задачи на разработку приложения информационной системы;
2. Моделирование бизнес-процессов и построение концептуальной модели базы данных информационной системы;
3. Логическое и физическое моделирование базы данных информационной системы с применением современных case-средств;
4. Развертывание базы данных информационной системы на сервере целевой СУБД и наполнение ее тестовыми данными;
5. Разработка и отладка программных средств доступа к базе данных при реализации автоматизируемых бизнес-процессов;
6. Тестирование разработанных программных средств.

2. Графическая часть

Пояснительная записка должна включать:

- диаграмму и спецификации вариантов использования;
- диаграммы «сущность-связь» для различных уровней моделей БД;
- архитектурные диаграммы для разрабатываемых программных средств;
- диаграммы деятельности для описания сценариев и алгоритмов работы отдельных компонентов разрабатываемых программных средств.

3. Экспериментальная часть

1. Разработка, развертывание и наполнение тестовыми данными базы данных ИС;
2. Разработка, отладка и тестирование программных средств ведения данных в контексте автоматизируемых бизнес-процессов для заданной категории пользователей.

Срок выполнения проекта по разделам

1	Анализ предметной области и постановка задачи на разработку приложения ИС	к 20 сентября	2025 г.
2	Моделирование бизнес-процессов и разработка концептуальной модели БД	к 4 октября	2025 г.
3	Логическое и физическое моделирование БД ИС	к 18 октября	2025 г.
4	Развертывание и наполнение тестовыми данными БД ИС	к 25 октября	2025 г.
5	Разработка, отладка и тестирование программного обеспечения доступа к данным в контексте автоматизируемых бизнес-процессов	к 10 декабря	2025 г.
6	Оформление пояснительной записки	к 14 декабря	2025 г.
7	Защита курсового проекта	к 16 декабря	2025 г.

Дата выдачи задания 5 сентября 2025 г.

Дата защиты проекта 17.12.2025 г.

Руководитель Шибанов /С.В. Шибанов/

Задание получил _____ 5 сентября 2025 г.

Студент Гришин /М. М. Гришин/

Реферат

Пояснительная записка содержит 143 листа, 17 рисунков, 53 таблицы, 4 использованных источника, 6 приложений.

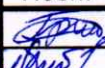
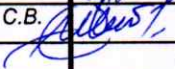
БАЗА ДАННЫХ, ИНФОРМАЦИОННАЯ СИСТЕМА, СУБД, POSTGRESQL, ПРОЦЕДУРА, ФУНКЦИЯ ТРИГГЕР, ПРЕДМЕТНАЯ ОБЛАСТЬ, ИНФОРМАЦИОННАЯ СИСТЕМА.

Объект разработки Программные средства ведения данных библиотекаря в единой базе данных информационного сервиса сети городских библиотек.

Цель разработки: изучить предметную область и провести её анализ, выделить бизнес-процессы, разработать концептуальную, логическую и физическую модель базы данных, а также разработать и протестировать программные средства доступа к базе данных.

Результаты разработки: исследована и проанализирована предметная область, выделены основные бизнес-процессы, разработана база данных и программные средства доступа к базе данных.

Средства разработки: СУБД PostgreSQL, Visual Studio 2022, язык программирования C#.

					ПГУ 09.03.04 – 7КР222.05 ПЗ		
Изм.	Лист	№ докум.	Подп.	Дата			
Разраб.		Гришин М.М.			Лит.	Лист	Листов
Пров.		Шибанов С.В.		17.12.25		4	143
Н.контр.					22ВП2		
Утв.		Козлов. А. Ю.					

Содержание

Введение	6
1. Анализ предметной области информационной системы и разработка требований к программным средствам	7
1.1. Анализ предметной области информационной системы	7
1.2. Планирование разработки информационной системы	7
1.3. Анализ функциональных возможностей программных средств	8
1.4. Разработка концептуальной схемы базы данных	11
2. Моделирование и развертывание базы данных	14
2.1. Логическое моделирование базы данных	14
2.2. Физическое моделирование базы данных	29
2.3. Развертывание и наполнение базы данных тестовыми данными	35
2.3.1. Развертывание базы данных	35
2.3.2. Наполнение базы данных тестовыми данными	37
3. Разработка и тестирование программных средств ведения данных	40
3.1. Архитектура и состав программных средств ведения данных	40
3.2. Разработка программных средств ведения данных	41
Тестирование программных средств ведения данных	43
Заключение	49
Список использованных источников	50
Приложение А Спецификации концептуальной схемы базы данных	51
Приложение Б Спецификации логической схемы базы данных	63
Приложение В Спецификации физической схемы базы данных	75
Приложение Г Скрипты (исходные коды) для создания объектов базы данных	112
Приложение Д Скрипты (исходные коды) для наполнения базы данных тестовыми данными	124
Приложение Е Исходные коды программных средств ведения данных	134

Введение

Современные библиотеки трансформируются из простых хранилищ книг в многофункциональные информационные центры. Эффективное управление такой структурой невозможно без автоматизации ключевых процессов: учета читателей, управления книжным фондом, контроля выдачи и возврата литературы.

Целью данной курсовой работы является разработка программных средств ведения данных библиотекаря в составе единой информационной системы сети городских библиотек.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ предметной области «Сеть городских библиотек» и выявить требования к системе со стороны библиотекаря.
2. Спроектировать базу данных, обеспечивающую хранение и целостность информации.
3. Реализовать физическую модель базы данных на СУБД PostgreSQL.[1,2]
4. Разработать запросы и алгоритмы для автоматизации рутинных операций библиотекаря.

Объектом исследования является процесс обслуживания читателей в городской библиотеке. Предметом разработки — программные средства и база данных для рабочего места библиотекаря.

1. Анализ предметной области информационной системы и разработка требований к программным средствам

1.1. Анализ предметной области информационной системы

Организационная структура библиотеки включает в себя:

Читателей: Регистрируется, заказывает и возвращает книги.

Библиотекарей: сотрудники, непосредственно взаимодействующие с читателями: выдают и принимают книги, формируют заявки на пополнение или замену фонда.

Работников книжного фонда: отвечают за состояние фонда, принимают заявки от библиотекарей на приобретение или замену книг, инициируют утилизацию, взаимодействуют с менеджером по поставкам.

Менеджера по поставкам: осуществляет заказ книг у поставщиков на основе заявок от работника книжного фонда.

Перечень категорий заинтересованных лиц (потенциальных пользователей ИС):

Наименование категории	Роль в предметной области	Наименование категории
Читатели	Основные потребители услуг библиотеки. Регистрируются в системе, осуществляют заказ книг, возвращают их.	Читатели
Библиотекари	Обслуживают читателей: выдают и принимают книги, фиксируют их состояние, формируют заявки для работника книжного фонда.	Библиотекари

1.2. Планирование разработки информационной системы

Общая концепция проекта информационной системы:

- **Наименование системы:** Информационная система «Сеть городских библиотек».
- **Категория системы:** Корпоративная информационная система.
- **Целевая аудитория:** Читатели, Библиотекари, Работники книжного фонда, Менеджер по поставкам.

- **Основные функции:** Ведение электронного каталога книг, учет читателей и их заказов, управление процессом выдачи и возврата.

- **Ключевое преимущество:** Повышение скорости и качества обслуживания читателей, снижение количества ошибок в учете.

Границы проекта ИС:

- Учет читателей и ведение электронных читательских билетов.
- Управление библиотечным фондом (учет экземпляров).
- Автоматизация процессов выдачи и возврата литературы.
- Генерация операционных отчетов и аналитики.

Состав приложений и классы пользователей:

- **Приложение:** Рабочее место библиотекаря (АРМ Библиотекаря).
- **Пользователь:** Библиотекарь.

Профиль пользователя «Библиотекарь»:

- **Основная ценность:** Автоматизация рутинных операций, минимизация ошибок, быстрый доступ ко всей необходимой информации для обслуживания читателей.

- **Ключевые функции:** Обслуживание выдачи/возврата, работа с читателями, формирование оперативных заявок.

1.3. Анализ функциональных возможностей программных средств

Многоуровневая модель для класса пользователей «Библиотекарь» (Приложение: АРМ Библиотекаря):

Уровень бизнес-процессов:

- Оформление выдачи книги;
- Оформление возврата книги;
- Анализ работы библиотеки.

Уровень бизнес-операций:

- Проверка состояния книги;
- Просмотр отчёта по популярным книгам;

- Просмотр количества читателей, не сдавших книги в срок, по месяцам.

Уровень операций с данными:

- Проверка наличия книги;
- Подтверждение выдачи книги;
- Чтение данных читателя;
- Подтверждение возврата;
- Выписывание штрафа;
- Получение информации о просроченной сдаче книг.

На основе анализа требований была разработана диаграмма вариантов использования (Use Case Diagram) для роли «Библиотекарь». Диаграмма вариантов использования представлена на рисунке 1.

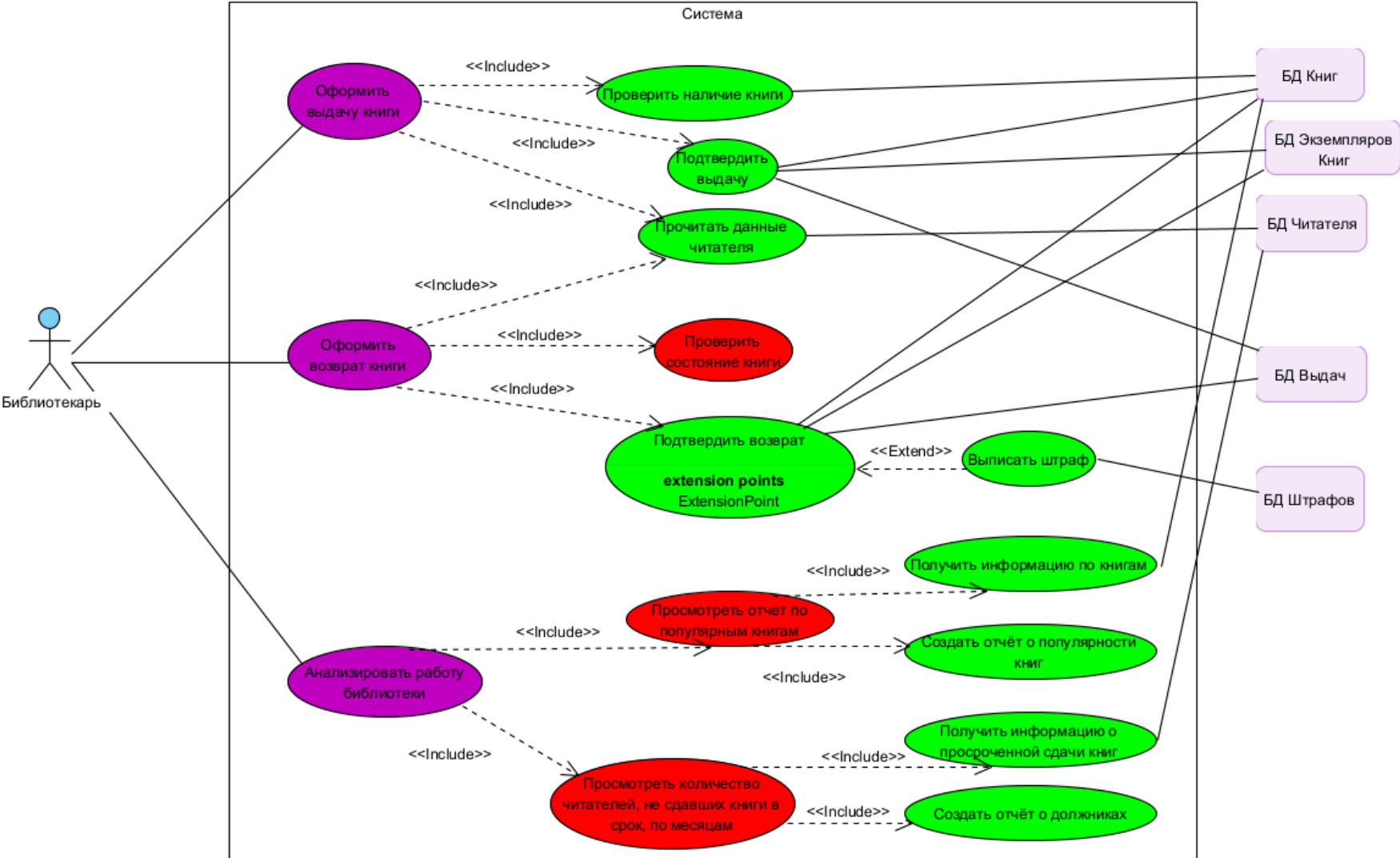


Рисунок 1 – Диаграмма вариантов использования для роли «Библиотекарь»

Спецификация варианта использования «Обслуживание читателя (Выдача и возврат)»:

- **Актеры:** Библиотекарь (Основной), Читатель, Система.
- **Цель:** Эффективно и корректно выполнить операции выдачи и возврата литературы.
- **Основной поток:** Библиотекарь сканирует билет, система отображает профиль. При выдаче сканирует книгу, система фиксирует выдачу. При возврате проверяет состояние и дату, при необходимости начисляет штраф.

1.4. Разработка концептуальной схемы базы данных

Проанализировав бизнес-процессы, был выделен следующий первоначальный набор ключевых информационных объектов (сущностей):

1. **Читатель** (Зарегистрированные пользователи).
2. **Выдача** (Запросы читателей на выдачу книг).
3. **Книга** (Издания в каталоге).
4. **Экземпляр книги** (Конкретные физические копии).
5. **Штраф** (Начисления за просрочку).
6. **Библиотекарь** (Работники библиотеки).

Детальный анализ CRUD-матрицы для Библиотекаря:

- **Читатель:** R (Read) — Просмотр данных для идентификации.
- **Выдача:** R, U (Read, Update) — Просмотр активных выдач, обновление статуса (выдана/возвращена).
- **Штраф:** C, R, U — Начисление штрафа, просмотр и отметка об оплате.

На рисунке 2 предоставлена диаграмма концептуальной схемы базы данных.

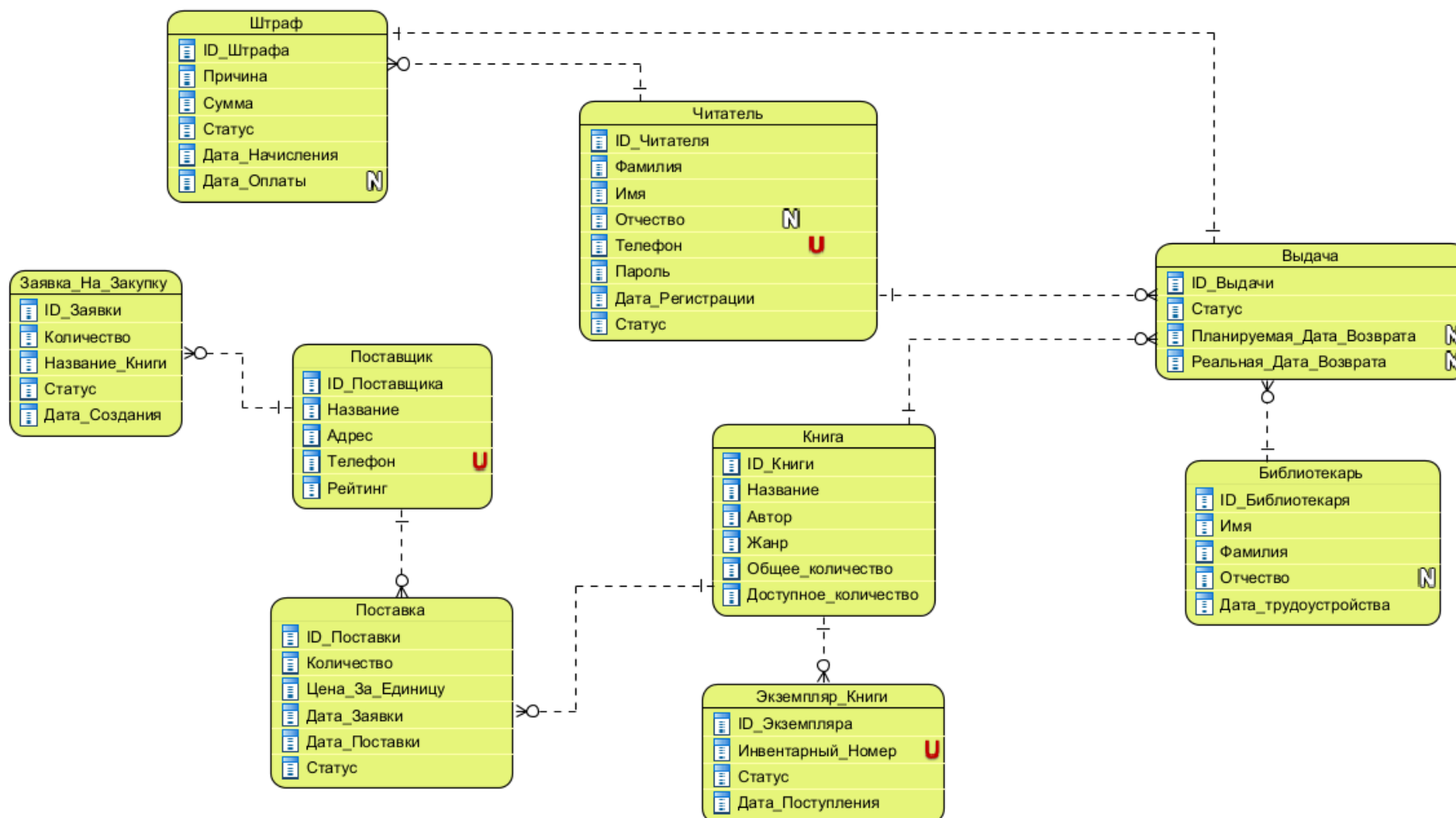


Рисунок 2 – Диаграмма концептуальной схемы базы данных

Были составлены спецификации концептуальной модели базы данных включающие:

- спецификации сущностей;
- спецификации атрибутов для каждой сущности;
- спецификации связей между сущностями.

Результат предоставлен в приложении А.

2. Моделирование и развертывание базы данных

2.1. Логическое моделирование базы данных

В ходе разработки логической схемы реляционной базы данных выделили предварительный набор отношений, атрибутов и связей между отношениями на основе правил отображения элементов концептуальной схемы в элементы логической схемы базы данных[3,4]. Предварительная логическая модель предоставлена на рисунке 3.

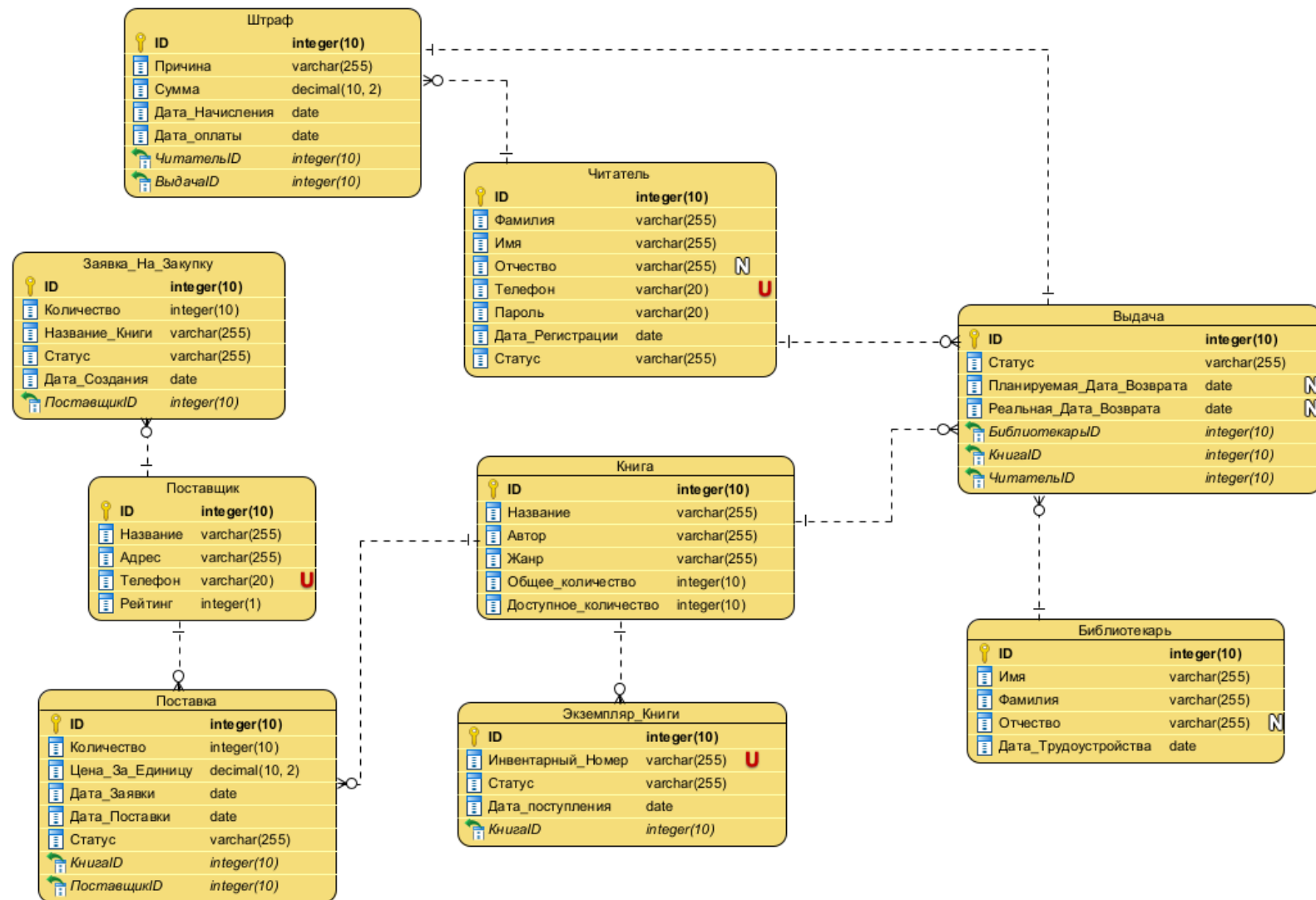


Рисунок 3 – Предварительная диаграмма логической схемы базы данных

Спецификации функциональных зависимостей предоставлены в таблицах 1-9.

Таблица 1 – Функциональные зависимости отношения «Читатель»

№	Символ	Функциональная зависимость	Бизнес-правило	Тип ФЗ	Аномалии
1	f1	ID → фамилия, имя, отчество, телефон, пароль дата_регистрации, статус	Каждый читатель уникально определяется своим идентификатором	Полная	Есть (Статус)
2	f2	Телефон → ID	Телефон уникален для каждого читателя	Полная	Нет

Таблица 2 – Функциональные зависимости отношения «Книга»

№	Символ	Функциональная зависимость	Бизнес-правило	Тип ФЗ	Аномалии
1	f1	ID → название, автор, жанр, общее_количество, доступное_количество	Каждая книга уникально определяется своим идентификатором. Атрибут «Автор» неатомарен и допускает multiple авторов	Полная	Есть (автор – неатомарность, жанр – транзитивная зависимость)

Таблица 3 – Функциональные зависимости отношения «Экземпляр_Книги»

№	Символ	Функциональная зависимость	Бизнес-правило	Тип ФЗ	Аномалии
1	f1	ID → КнигаID, Инвентарный_Номер, статус, дата_поступления	Каждый экземпляр книги уникально определяется своим идентификатором	Полная	Есть (Статус)
2	f2	Инвентарный_Номер → ID	Инвентарный номер уникален для каждого экземпляра книги	Полная	Нет

Таблица 4 – Функциональные зависимости отношения «Библиотекарь»

№	Символ	Функциональная зависимость	Бизнес-правило	Тип ФЗ	Аномалии
1	f1	ID → фамилия, имя, отчество, Дата_трудоустройства	Каждый библиотекарь уникально определяется своим идентификатором	Полная	Нет

Таблица 5 – Функциональные зависимости отношения «Штраф»

№	Символ	Функциональная зависимость	Бизнес-правило	Тип ФЗ	Аномалии
1	f1	ID → причина, сумма, дата_начисления, дата_оплаты, читательID, выдачаID, статус	Каждый штраф уникально определяется своим идентификатором	Полная	Есть (Причина и статус)

Таблица 6 – Функциональные зависимости отношения «Выдача»

№	Символ	Функциональная зависимость	Бизнес-правило	Тип ФЗ	Аномалии
1	f1	ID → статус, планируемая_дата_возврата, реальная_дата_возврата, библиотекарьID, книгаID, читательID	Каждая выдача уникально определяется своим идентификатором. Структура не позволяет учитывать несколько книг в одной операции	Полная	Есть (Статус и структурная – нельзя указать несколько книг в одной выдаче)

Таблица 7 – Функциональные зависимости отношения «Поставщик»

№	Символ	Функциональная зависимость	Бизнес-правило	Тип ФЗ	Аномалии
1	f1	ID → название, адрес, телефон, рейтинг	Каждый поставщик уникально определяется своим идентификатором	Полная	Есть (адрес)
2	f2	Телефон → ID	Телефон уникален для каждого поставщика	Полная	Нет

Таблица 8 – Функциональные зависимости отношения «Заявка_На_Закупку»

№	Символ	Функциональная зависимость	Бизнес-правило	Тип ФЗ	Аномалии
1	f1	ID → количество, название_книги, статус, дата_создания, поставщикID	Каждая заявка на закупку уникально определяется своим идентификатором	Полная	Есть (Статус и структурная – нельзя указать несколько книг в одной заявке)

Таблица 9 – Функциональные зависимости отношения «Поставка»

№	Символ	Функциональная зависимость	Бизнес-правило	Тип ФЗ	Аномалии
1	f1	ID → количество, цена_за_единицу, статус, дата_заявки, дата_поставки, поставщикID, КнигаID	Каждая поставка уникально определяется своим идентификатором. Структура не позволяет учитывать несколько книг в одной операции	Полная	Есть (Статус и структурная – нельзя указать несколько книг в одной поставке)

В ходе анализа функциональных зависимостей предварительного набора отношений выявлены следующие аномалии:

1. **Отношение «Книга»** - нарушение 1НФ (неатомарность атрибута «Автор»), транзитивные зависимости (атрибут «Жанр»)
2. **Отношение «Выдача»** - структурная аномалия (невозможность учета нескольких книг в одной операции выдачи)
3. **Отношение «Поставка»** - структурная аномалия (невозможность учета нескольких книг в одной поставке)
4. **Отношение «Поставщик»** - неатомарность атрибута «Адрес»
5. Наличие неатомарных атрибутов-статусов в различных отношениях, требующих вынесения в справочники.
6. Отсутствие сущности «Библиотека» для учета филиалов сети.
7. Отсутствие сущности «Должность» для классификации сотрудников.
8. Отсутствие сущности для учета внутренних заявок («Заявка_Менеджеру»).
9. Наличие атрибута «Название_Книги» в отношениях «Заявка_На_Закупку» и «Позиция_Поставки» вместо ссылки на основную сущность «Книга», что ведет к избыточности и возможной

несогласованности данных.

Подлежат декомпозиции следующие отношения:

1. **«Книга»** → декомпозиция на 3 отношения
2. **«Выдача»** → декомпозиция на 2 отношения
3. **«Поставка»** → декомпозиция на 2 отношения
4. **«Поставщик»** → нормализация атрибутов
5. **Библиотекарь** → замена на сущность **«Сотрудник»** с добавлением связей
6. Все **атрибуты-статусы** → вынесение в отдельные справочные сущности
7. **«Позиция_Поставки»** → замена атрибута **«Название_Книги»** на внешний ключ **«ID_Книги»**
8. **«Заявка_На_Закупку»** → декомпозиция на 2 отношения
9. Добавление новых сущностей: **«Библиотека»**, **«Должность»**, **«Заявка_Менеджеру»**.

Доработка модели для учета сети библиотек и справочной информации

Основание для доработки:

- Модель должна отражать принадлежность к сети городских библиотек, подразумевающей наличие нескольких филиалов (**Библиотека**).
- Атрибуты-статусы, хранящиеся в различных отношениях, являются неатомарными и ограниченными. Их вынесение в справочники (**Статус_...**) повышает гибкость системы (позволяет добавлять новые статусы без изменения структуры БД) и устраняет избыточность.
- Сущность **Библиотекарь** не учитывает других должностей (менеджер, администратор и т.д.). Замена на **Сотрудник** с привязкой к **Должности** и **Библиотеке** унифицирует учет кадров.
- Для внутренней работы библиотечной сети необходим механизм постановки и контроля задач (**Заявка_Менеджеру**).

– Связь заявок на закупку и поставок с основным каталогом **Книга** через внешний ключ, а не через текстовое поле, обеспечивает целостность данных и исключает дублирование.

Процесс доработки:

1. Создание независимых сущностей-справочников: **Библиотека**, **Должность**.

2. Создание сущностей-справочников для всех статусов: **Статус_Читателя**, **Статус_Выдачи**, **Статус_Позиции_Выдачи**, **Статус_Экземпляра**, **Статус_Заявки_На_Закупку**, **Статус_Поставки**, **Статус_Заявки_Менеджеру**.

3. Замена сущности Библиотекарь на **Сотрудник** с добавлением внешних ключей **ID_Библиотеки** и **ID_Должности**.

4. Создание сущности **Заявка_Менеджеру** для учета внутренних поручений.

5. Замена атрибута Название_Книги на **ID_Книги** (FK) в отношениях **Заявка_На_Закупку** и **Позиция_Поставки**.

6. Замена текстовых полей Статус на соответствующие внешние ключи **ID_Статуса** во всех отношениях, где они присутствуют.

Результаты доработки:

Новая сущность: «**Библиотека**»

Библиотека (
ID INTEGER PK,
Название VARCHAR(255) NOT NULL,
Улица VARCHAR(500) NOT NULL,
Дом INTEGER(10) NOT NULL,
Телефон VARCHAR(20)
)

Новая сущность: «**Должность**»

Должность (
ID INTEGER PK,
Название VARCHAR(100) NOT NULL

)

Новая сущность: «**Заявка_Менеджеру**»

Заявка_Менеджеру (

ID INTEGER PK,

ID_Сотрудника INTEGER FK NOT NULL, -- Кто создал заявку

ID_Книги INTEGER FK NOT NULL,

Дата_Создания DATE NOT NULL,

ID_Статуса INTEGER FK NOT NULL -- Ссылка на справочник статусов

)

Пример новых сущностей-справочников (структура одинакова для всех):

Новая сущность: «**Статус_Читателя**»

Статус_Читателя (

ID INTEGER PK,

Название VARCHAR(50) NOT NULL -- Например: 'Активен'

)

Измененная сущность: «**Сотрудник**» (заменяет «Библиотекарь»)

Сотрудник (

ID INTEGER PK,

Фамилия VARCHAR(100) NOT NULL,

Имя VARCHAR(100) NOT NULL,

Отчество VARCHAR(100),

ID_Библиотеки INTEGER FK NOT NULL,

ID_Должности INTEGER FK NOT NULL,

Дата_Трудоустройства DATE NOT NULL,

)

Измененная сущность: «**Читатель**»

Читатель (

...

ID_Статуса INTEGER FK NOT NULL, -- Заменяет атрибут 'Статус'

ID_Библиотеки_Регистрации INTEGER FK NOT NULL -- Читатель привязан к библиотеке, где зарегистрирован

)

Измененная сущность: «Позиция_Поставки»

Позиция_Поставки (

...

ID_Книги INTEGER FK NOT NULL -- Заменяет атрибут 'Название_Книги'

)

(Аналогичные изменения по замене статусов на FK должны быть проведены для отношений: Выдача, Позиция_Выдачи, Экземпляр_Книги, Поставка, Штраф)

Декомпозиция отношения «Книга»

Основание для декомпозиции:

- Нарушение 1НФ: атрибут «Автор» неатомарен, допускает multiple авторов
- Транзитивная зависимость: атрибут «Жанр» может иметь собственные атрибуты
- Связь «многие-ко-многим» между книгами и авторами

Процесс декомпозиции:

1. Выделение сущности «Автор» для нормализации данных об авторах
2. Создание ассоциативной таблицы «Книга_Автор» для разрешения связи M:N
3. Выделение сущности «Жанр» для устранения транзитивной зависимости

Результаты декомпозиции:

Новое отношение: «Автор»

Автор (

ID INTEGER PK,

Фамилия VARCHAR(100) NOT NULL,

Имя VARCHAR(100) NOT NULL,

Отчество VARCHAR(100)

)

Новое отношение: «Жанр»

Жанр (
 ID INTEGER PK,
 Название VARCHAR(50) NOT NULL
)

Новое отношение: «Книга_Автор»

Книга_Автор (
 ID_Книги INTEGER PK/FK,
 ID_Автора INTEGER PK/FK
)

Измененное отношение: «Книга»

Книга (
 ID INTEGER PK,
 Название VARCHAR(255) NOT NULL,
 ID_Жанра INTEGER FK, -- вместо атрибута «Жанр»
 Общее_Количество INTEGER NOT NULL,
 Доступное_Количество INTEGER NOT NULL
)

Декомпозиция отношения «Выдача»

Основание для декомпозиции:

- Структурная аномалия: невозможность учета нескольких книг в одной операции выдачи
- Избыточность данных: повторение информации о читателе и библиотекаре для каждой книги
- Нарушение 1НФ: при попытке хранить multiple книг в одной записи

Процесс декомпозиции:

1. Создание отношения «Позиция_Выдачи» для учета отдельных книг
2. Разделение атрибутов на заголовочные и позиционные

Результаты декомпозиции:

Измененное отношение: «Выдача»

Выдача (

ID INTEGER PK,

ЧитательID INTEGER FK NOT NULL,

БиблиотекарьID INTEGER FK NOT NULL,

Дата_Запроса DATE NOT NULL,

Статус VARCHAR(20) NOT NULL,

Планируемая_Дата_Возврата DATE

)

Новое отношение: «Позиция_Выдачи»

Позиция_Выдачи (

ID INTEGER PK,

ID_Выдачи INTEGER FK NOT NULL,

ID_Книги INTEGER FK NOT NULL,

Статус_Позиции VARCHAR(20) NOT NULL,

Реальная_Дата_Возврата DATE

)

Декомпозиция отношения «Поставка»

Основание для декомпозиции:

- Структурная аномалия: невозможность учета нескольких книг в одной операции выдачи
- Избыточность данных: повторение информации о поставщике и датах для каждой книги
- Нарушение принципов нормализации

Процесс декомпозиции:

1. Разделение на заголовок поставки и позиции поставки
2. Выделение общих атрибутов в заголовок

Результаты декомпозиции:

Измененное отношение: «Поставка»

Поставка (

ID INTEGER PK,
 ID_Поставщика INTEGER FK NOT NULL,
 Дата_Заявки DATE NOT NULL,
 Дата_Поставки DATE NOT NULL,
 Статус VARCHAR(20) NOT NULL,
 Общая_Стоимость DECIMAL(10,2)
)

Новое отношение: «Позиция_Поставки»

Позиция_Поставки (
 ID INTEGER PK,
 ID_Поставки INTEGER FK NOT NULL,
 Название_Книги VARCHAR(255) NOT NULL,
 Количество INTEGER NOT NULL,
 Цена_За_Единицу DECIMAL(10,2) NOT NULL
)

Декомпозиция отношения «Заявка_На_Закупку»

Основание для декомпозиции:

- Структурная аномалия: невозможность учета нескольких книг в одной заявки
- Избыточность данных: повторение информации о поставщике
- Нарушение принципов нормализации

Процесс декомпозиции:

1. Разделение на заголовок поставки и позиции поставки
2. Выделение общих атрибутов в заголовок

Результаты декомпозиции:

Измененное отношение: «Заявка_На_Закупку»

Заявка_На_Закупку (
 Дата_Создания DATE NOT NULL,
 ID_Поставщика INTEGER FK NOT NULL,
 ID_Статуса INTEGER FK NOT NULL
)

Новое отношение: «Позиция_Заявки_На_Закупку»

```

Позиция_ Заявки_ На_ Закупку(
    ID INTEGER PK,
    Количество INTEGER NOT NULL,
    ID_Книги INTEGER FK NOT NULL,
    ID_Заявки_на_Закупку INTEGER FK NOT NULL,
    ID_Статуса INTEGER FK NOT NULL
)

```

Нормализация отношения «Поставщик»

Основание для нормализации:

- Неатомарность атрибута «Адрес»
- Составной характер адресных данных

Процесс нормализации:

1. Разделение атрибута «Адрес» на атомарные компоненты
2. Сохранение семантики адресной информации

Результаты нормализации:

Измененное отношение: «Поставщик»

```

Поставщик (
    ID_Поставщика INTEGER PK,
    Название VARCHAR(200) NOT NULL,
    Город VARCHAR(50) NOT NULL,      -- вместо «Адрес»
    Улица VARCHAR(100) NOT NULL,     -- вместо «Адрес»
    Дом VARCHAR(10) NOT NULL,        -- вместо «Адрес»
    Телефон VARCHAR(20) NOT NULL,
    Рейтинг INTEGER NOT NULL
)

```

После проведения всех мероприятий по устранению аномалий получен финальный набор из **29 отношений**:

1. Библиотека (новая)
2. Читатель
3. Книга (нормализованная)
4. Автор (новая)
5. Жанр (новая)

6. Книга_Автор (новая)
7. Экземпляр_Книги
8. Сотрудник (замена "Библиотекаря")
9. Поставщик (нормализованный)
10. Выдача (нормализованная)
11. Позиция_Выдачи (новая)
12. Заявка_На_Закупку
13. Позиция_Поставки_На_Закупку (новая)
14. Поставка (нормализованная)
15. Позиция_Поставки (новая)
16. Штраф
17. Должность (новая)
18. Заявка_Менеджеру (новая)
19. Статус_Заявки_Менеджеру (новая)
20. Статус_Читателя (новая)
21. Статус_Выдачи (новая)
22. Статус_Позиции_Выдачи (новая)
23. Статус_Позиции_Поставки (новая)
24. Статус_Экземпляра (новая)
25. Статус_Заявки_На_Закупку (новая)
26. Статус_Позиции_Заявки_На_Закупку (новая)
27. Статус_Поставки (новая)
28. Статус_Штрафа (новая)
29. Причина_Штрафа (новая)

Все отношения финального набора соответствуют **третьей нормальной форме (3НФ)**, аномалии модификации данных устранены.

Графическое представление логической схемы базы данных для финального набора отношений в виде диаграммы «сущность-связь» предоставлено на рисунке 4.

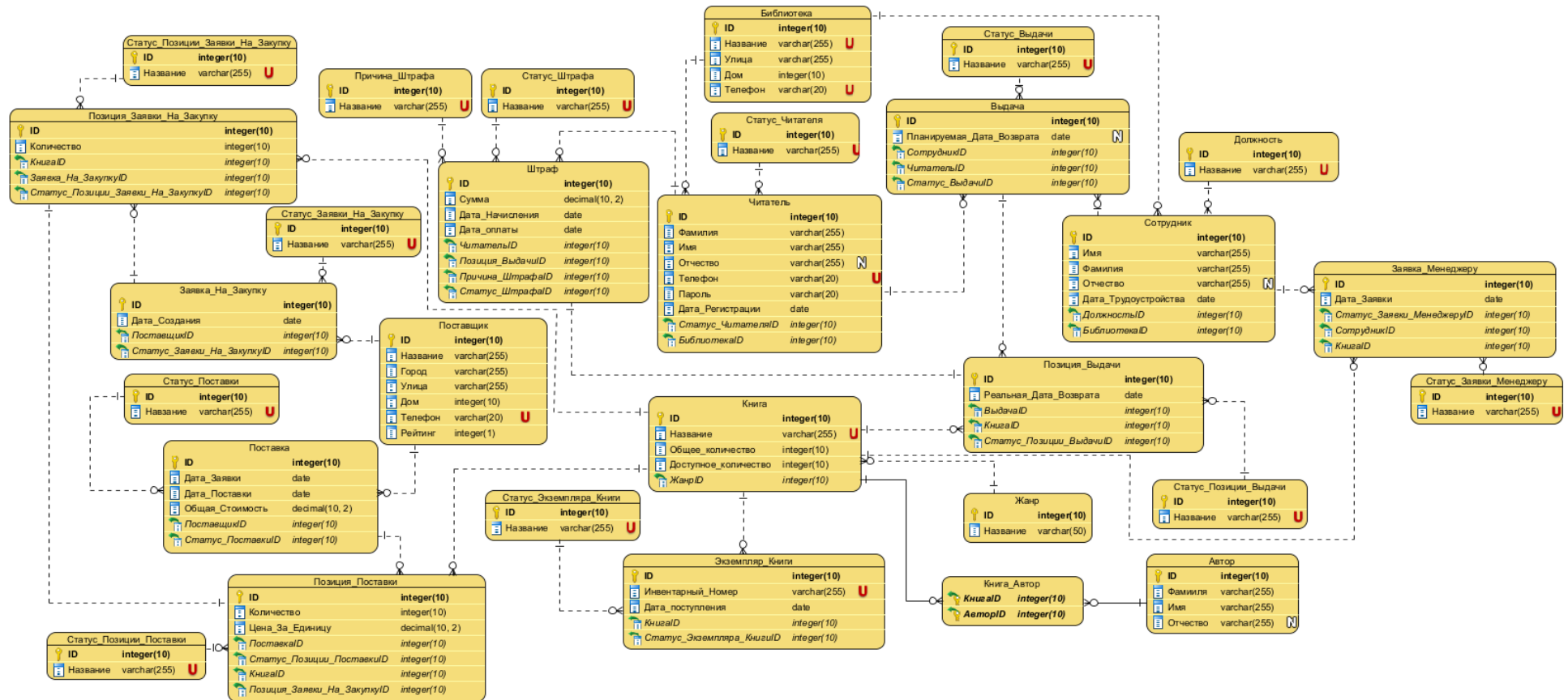


Рисунок 4 – Финальная диаграмма логической схемы базы данных

Спецификации логической схемы базы данных предоставлены в приложении Б.

2.2 Физическое моделирование базы данных

Для разработки физической модели базы данных была составлена диаграмма «сущность-связь»[4]. Физическая модель предоставлена на рисунке 5.

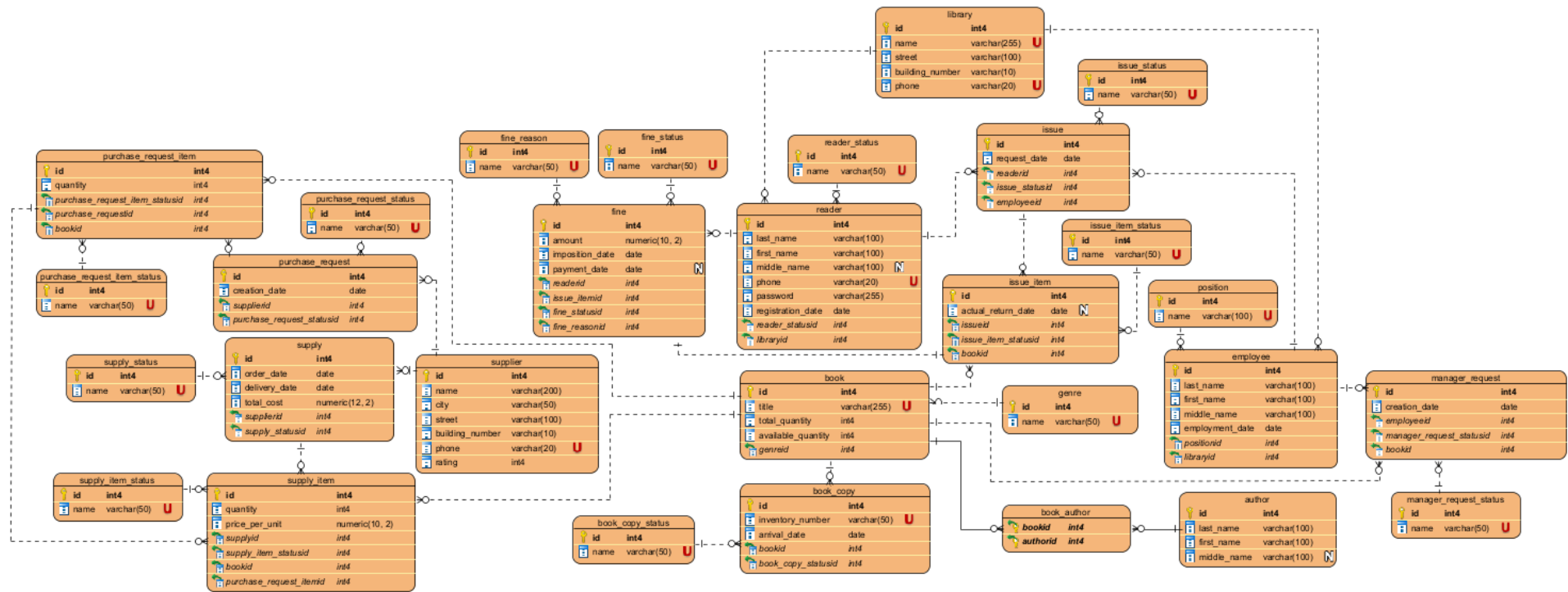


Рисунок 5 – Диаграмма физической схемы базы данных

Подробное описание таблиц (сущностей), их столбцов, индексов, ограничений и связей было создано при помощи приложения Visual Paradigm и предоставлено в приложении В.

На таблице 10 предоставлено описание связей между таблицами, включая ограничения ссылочной целостности для связи.

Таблица 10 – Описание связей между таблицами, включая ограничения ссылочной целостности для связи

Отношения связи	Обязательность	Мощность	Смысл связи		Тип	Ограничения целостности
			Parent to Child	Child to Parent		
library → employee	No Nulls	One-to-Many	В ней работают	Работает в библиотеке	Non-identifying	ON DELETE RESTRICT, ON UPDATE CASCADE
position → employee	No Nulls	One-to-Many	Занимают должность	Должность занимают	Non-identifying	ON DELETE RESTRICT, ON UPDATE CASCADE
library → reader	No Nulls	One-to-Many	В ней зарегистрированы	Зарегистрирован в библиотеке	Non-identifying	ON DELETE RESTRICT, ON UPDATE CASCADE
reader → issue	No Nulls	One-to-Many	Оформляет заказы	Оформлен читателем	Non-identifying	ON DELETE RESTRICT, ON UPDATE CASCADE
employee → issue	No Nulls	One-to-Many	Оформляет выдачи	Оформлен сотрудником	Non-identifying	ON DELETE RESTRICT, ON UPDATE CASCADE
issue → issue_item	No Nulls	One-to-Many	Содержит позиции	Входит в выдачу	Identifying	ON DELETE CASCADE, ON UPDATE CASCADE
issue_item → book	No Nulls	Many-to-One	Относится к книге	Выдана в позициях	Non-identifying	ON DELETE RESTRICT, ON UPDATE CASCADE
book → book_author	No Nulls	One-to-Many	Имеет авторов	Написана автором	Identifying	ON DELETE CASCADE, ON UPDATE CASCADE

Продолжение таблицы 10

Отношения связи	Обязательность	Мощность	Смысл связи		Тип	Ограничения целостности
			Parent to Child	Child to Parent		
author → book_author	No Nulls	One-to-Many	Написал книги	Автор книги	Identifying	ON DELETE CASCADE, ON UPDATE CASCADE
genre → book	No Nulls	One-to-Many	Содержит книги	Принадлежит жанру	Non-identifying	ON DELETE RESTRICT, ON UPDATE CASCADE
book → book_copy	No Nulls	One-to-Many	Имеет экземпляры	Является копией	Non-identifying	ON DELETE RESTRICT, ON UPDATE CASCADE
supplier → supply	No Nulls	One-to-Many	Осуществляет поставки	Поставлен поставщиком	Non-identifying	ON DELETE RESTRICT, ON UPDATE CASCADE
supply → supply_item	No Nulls	One-to-Many	Содержит позиции	Входит в поставку	Identifying	ON DELETE CASCADE, ON UPDATE CASCADE
purchase_request → purchase_request_item	No Nulls	One-to-Many	Содержит позиции	Входит в заявку	Identifying	ON DELETE CASCADE, ON UPDATE CASCADE
reader → fine	No Nulls	One-to-Many	Имеет штрафы	Начислен читателю	Non-identifying	ON DELETE RESTRICT, ON UPDATE CASCADE
issue_item → fine	No Nulls	One-to-Zero-or-One	Может повлечь штраф	Начислен за позицию	Non-identifying	ON DELETE RESTRICT, ON UPDATE CASCADE
book → purchase_request_item	No Nulls	One-to-Many	Запрашивается к покупке	Заявлена на закупку	Non-identifying	ON DELETE RESTRICT, ON UPDATE CASCADE
book → supply_item	No Nulls	One-to-Many	Поставляется в составе	Поставлен как книга	Non-identifying	ON DELETE RESTRICT, ON UPDATE CASCADE
employee → manager_request	No Nulls	One-to-Many	Назначен исполнителем	Назначена менеджеру	Non-identifying	ON DELETE RESTRICT, ON UPDATE CASCADE

Продолжение таблицы 10

Отношения связи	Обязат ельнос ть	Мощнос ть	Смысл связи		Тип	Ограничения целостности
			Parent to Child	Child to Parent		
reader_status → reader	No Nulls	One-to- Many	Определяет статус	Имеет статус	Non- identif ying	ON DELETE RESTRICT, ON UPDATE CASCADE
issue_status → issue	No Nulls	One-to- Many	Определяет статус	Имеет статус	Non- identif ying	ON DELETE RESTRICT, ON UPDATE CASCADE
issue_item_status → issue_item	No Nulls	One-to- Many	Определяет статус	Имеет статус	Non- identif ying	ON DELETE RESTRICT, ON UPDATE CASCADE
book_copy_status → book_copy	No Nulls	One-to- Many	Определяет статус	Имеет статус	Non- identif ying	ON DELETE RESTRICT, ON UPDATE CASCADE
purchase_request _status → purchase_request	No Nulls	One-to- Many	Определяет статус	Имеет статус	Non- identif ying	ON DELETE RESTRICT, ON UPDATE CASCADE
supply_status → supply	No Nulls	One-to- Many	Определяет статус	Имеет статус	Non- identif ying	ON DELETE RESTRICT, ON UPDATE CASCADE
supply_item_stat us → supply_item	No Nulls	One-to- Many	Определяет статус	Имеет статус	Non- identif ying	ON DELETE RESTRICT, ON UPDATE CASCADE
manager_request _status → manager_request	No Nulls	One-to- Many	Определяет статус	Имеет статус	Non- identif ying	ON DELETE RESTRICT, ON UPDATE CASCADE
fine_status → fine	No Nulls	One-to- Many	Определяет статус	Имеет статус	Non- identif ying	ON DELETE RESTRICT, ON UPDATE CASCADE
purchase_request _item_status → purchase_request _item	No Nulls	One-to- Many	Определяет статус	Имеет статус	Non- identif ying	ON DELETE RESTRICT, ON UPDATE CASCADE
fine_reason → fine	No Nulls	One-to- Many	Определяет причину	Имеет причину	Non- identif ying	ON DELETE RESTRICT, ON UPDATE CASCADE

Окончание таблицы 10

Отношения связи	Обязательность	Мощность	Смысл связи		Тип	Ограничения целостности
			Parent to Child	Child to Parent		
supplier → purchase_request	No Nulls	One-to-Many	Предлагается для закупки	Заявлен поставщик	Non-identifying	ON DELETE RESTRICT, ON UPDATE CASCADE
book → manager_request	No Nulls	One-to-Many	Имеет связанные заявки	Относится к книге	Non-identifying	ON DELETE RESTRICT, ON UPDATE CASCADE

Был произведен расчёт оценки размера базы данных по следующей методике:

1. Оцениваем количество записей для каждой таблицы
2. Рассчитываем размер одной строки (сумма байт всех столбцов)
3. Вычисляем размер данных = количество записей × размер строки
4. Оцениваем размер индексов $\approx 30\%$ от размера данных
5. Суммируем данные + индексы

Результаты расчётов оценки размера базы данных предоставлены в таблице 11.

Таблица 11 – оценка размера базы данных

Таблица	Кол-во записей	Размер строки (байт)	Данные (МБ)	Индексы (МБ)	Всего (МБ)
library	50	400	0.02	0.01	0.03
reader	10,000	250	2.38	0.71	3.09
book	50,000	300	14.31	4.29	18.60
author	20,000	250	4.77	1.43	6.20
genre	10	100	0.001	0.0003	0.0013
book_author	80,000	8	0.61	0.18	0.79
book_copy	150,000	150	21.46	6.44	27.90
employee	500	250	0.12	0.04	0.16
issue	20,000	100	1.91	0.57	2.48
issue_item	40,000	100	3.81	1.14	4.95
purchase_request	5,000	100	0.48	0.14	0.62
purchase_request_item	15,000	100	1.43	0.43	1.86
supply	3,000	150	0.43	0.13	0.56
supply_item	10,000	200	1.91	0.57	2.48

Окончание таблицы 11

Таблица	Кол-во записей	Размер строки (байт)	Данные (МБ)	Индексы (МБ)	Всего (МБ)
fine	2,000	150	0.29	0.09	0.38
manager_request	2,000	100	0.19	0.06	0.25
reader_status	2	100	0.0002	0.0001	0.0003
issue_status	4	100	0.0004	0.0001	0.0005
issue_item_status	2	100	0.0002	0.0001	0.0003
book_copy_status	3	100	0.0003	0.0001	0.0004
purchase_request_status	4	100	0.0004	0.0001	0.0005
purch_req_item_status	4	100	0.0004	0.0001	0.0005
supply_status	3	100	0.0003	0.0001	0.0004
supply_item_status	2	100	0.0002	0.0001	0.0003
manager_request_status	3	100	0.0003	0.0001	0.0004
fine_status	2	100	0.0002	0.0001	0.0003
fine_reason	2	100	0.0002	0.0001	0.0003
position	3	100	0.0003	0.0001	0.0004
supplier	100	400	0.04	0.01	0.05
Итого	407,685	-	54.16	16.25	70.41

Итоговая оценка:

- **Общий размер данных:** ≈ 54.16 МБ
- **Общий размер индексов:** ≈ 16.25 МБ
- **Общий размер БД:** ≈ 70.41 МБ

Скрипт на языке SQL для создания объектов базы данных, сгенерированный в Visual Paradigm под СУБД PostgreSQL предоставлен в приложении Г.

2.3 Развертывание и наполнение базы данных тестовыми данными

2.3.1 Развертывание базы данных

Для развёртывания базы данных был выполнен ряд действий:

Выполнен вход в pgAdmin4 и создали базу данных city_libraries_db (Рисунок 6).

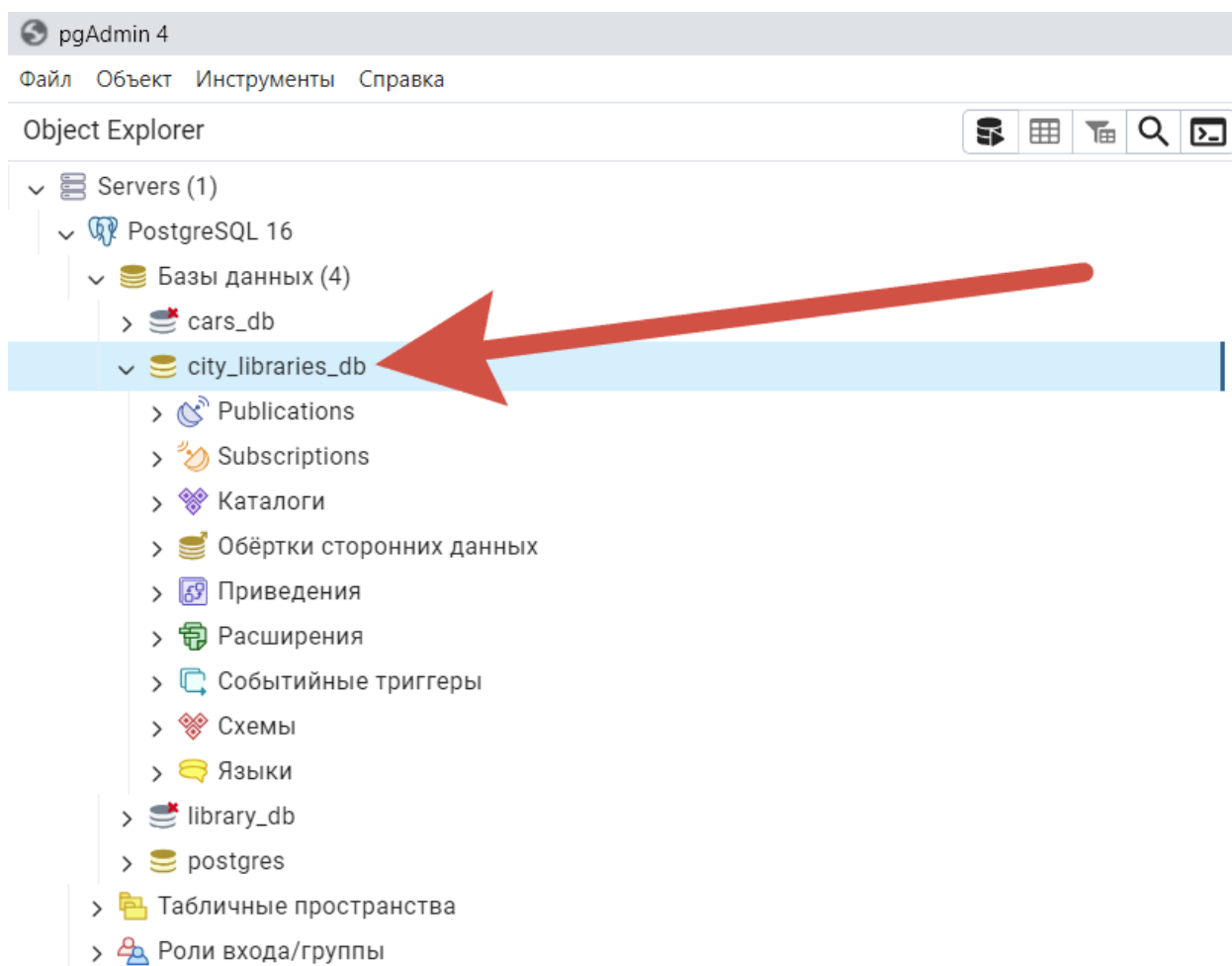


Рисунок 6 – Созданная база данных

Для создания таблиц, ограничений и индексов был использован ранее сгенерированный SQL-скрипт из приложения Г. Результат выполнения данного SQL-скрипта предоставлен на рисунке 7.

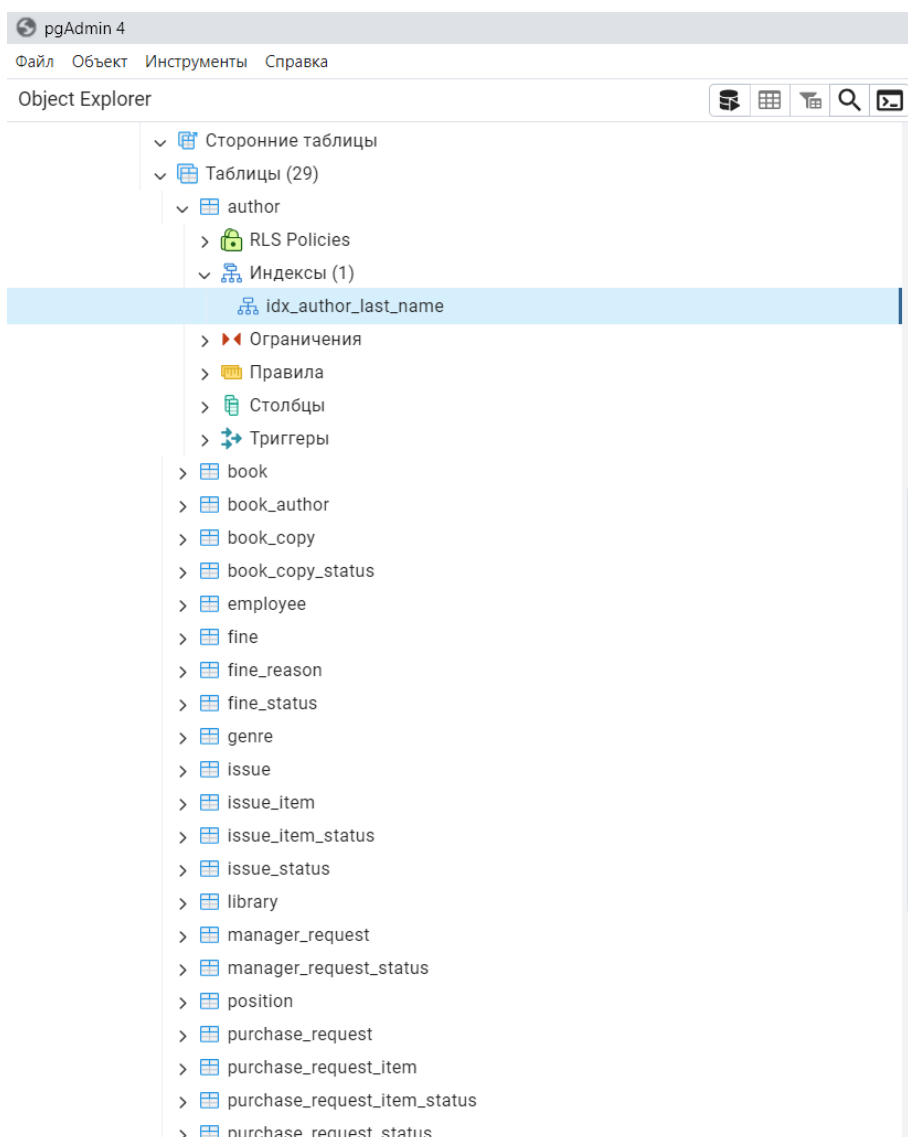


Рисунок 7 – Результат выполнения SQL-скрипта для создания таблиц, ограничений и индексов

2.3.2 Наполнение базы данных тестовыми данными

Для заполнения базы данных тестовыми данными был разработан SQL-скрипт, который:

1. Заполняет справочные таблицы фиксированными значениями
2. Генерирует тестовые данные для основных таблиц с использованием циклов
3. Обеспечивает ссылочную целостность данных

Данный код предоставлен в Приложении Д. Результат запуска файла для создания тестовых данных предоставлен на рисунке 8.

The screenshot shows a PostgreSQL client window titled "Тестовые данные.sql". The connection is to "city_libraries_db/postgres@PostgreSQL 16". The SQL script being executed is as follows:

```

1  SET session_replication_role = 'replica';
2
3  -- =====
4  -- БЛОК 1: ТАБЛИЦА LIBRARY (БИБЛИОТЕКИ)
5  -- =====
6  INSERT INTO library (name, street, building_number, phone)
7  SELECT
8      'Библиотека №' || i,
9      CASE (i % 10)
10         WHEN 0 THEN 'Ленина'

```

The output section, titled "Сообщения", displays the following messages:

```

ЗАМЕЧАНИЕ: Все 29 таблиц успешно заполнены!
ЗАМЕЧАНИЕ: =====
ЗАМЕЧАНИЕ: СТАТИСТИКА ЗАПОЛНЕНИЯ БАЗЫ ДАННЫХ
ЗАМЕЧАНИЕ: =====
ЗАМЕЧАНИЕ: Библиотеки: 50
ЗАМЕЧАНИЕ: Читатели: 10000
ЗАМЕЧАНИЕ: Книги (каталог): 50000
ЗАМЕЧАНИЕ: Авторы: 20000
ЗАМЕЧАНИЕ: Экземпляры книг: 150000
ЗАМЕЧАНИЕ: Сотрудники: 500
ЗАМЕЧАНИЕ: Выдачи: 20000
ЗАМЕЧАНИЕ: Позиции выдач: 40000
ЗАМЕЧАНИЕ: Связи книги-авторы: 79996
ЗАМЕЧАНИЕ: Заявки на закупку: 5000
ЗАМЕЧАНИЕ: Позиции заявок: 15000
ЗАМЕЧАНИЕ: Поставки: 3000
ЗАМЕЧАНИЕ: Позиции поставок: 10000
ЗАМЕЧАНИЕ: Штрафы: 2000
ЗАМЕЧАНИЕ: Заявки менеджера: 2000

```

Рисунок 8 – Результат запуска тестового файла

С помощью системных средств целевой СУБД были оценены реальный размер дискового пространства, занимаемого каждой таблицей (записи таблицы, индексы, системная информация и пр.) и базой данных в целом. Результат фактических данных размера занимаемого дискового пространства, занимаемого каждой таблицей и базой данных в целом предоставлен на рисунке 9.

	Таблица text	Кол-во записей numeric	Размер строки (байт) numeric	Данные (МБ) numeric	Индексы (МБ) numeric	Всего (МБ) numeric
18	issue_status	4	2048.00	0.01	0.03	0.04
19	issue_item_status	2	4096.00	0.01	0.03	0.04
20	book_copy_status	3	2730.67	0.01	0.03	0.04
21	purchase_request_status	4	2048.00	0.01	0.03	0.04
22	purchase_request_item_status	4	2048.00	0.01	0.03	0.04
23	supply_status	3	2730.67	0.01	0.03	0.04
24	supply_item_status	2	4096.00	0.01	0.03	0.04
25	manager_request_status	3	2730.67	0.01	0.03	0.04
26	fine_status	2	4096.00	0.01	0.03	0.04
27	fine_reason	2	4096.00	0.01	0.03	0.04
28	position	3	2730.67	0.01	0.03	0.04
29	genre	10	819.20	0.01	0.03	0.04
30	ИТОГО	407690	[null]	22.77	33.97	56.73

Рисунок 9 – Фактическая оценка базы данных

Сравнение фактической и расчётной таблицы оценки размеров показало, что в предыдущем подсчёте данные были выявлены не верно!

3. Разработка и тестирование программных средств ведения данных

В данном разделе описывается процесс программной реализации серверной части приложения для автоматизации рабочего места библиотекаря. Реализация выполнена в соответствии с диаграммой вариантов использования и спроектированной схемой базы данных.

3.1 Архитектура и состав программных средств ведения данных

Разработка велась на языке программирования **C#** с использованием платформы **.NET 8** и среды разработки **Visual Studio 2022**. В качестве архитектурного паттерна выбран **MVC (Model-View-Controller)** в реализации Web API.

Архитектура приложения состоит из следующих уровней:

1. **Уровень представления (Presentation Layer):** Реализован через **Swagger UI**, который предоставляет графический интерфейс для взаимодействия с API-методами (отправка запросов, просмотр ответов в формате JSON).
2. **Уровень бизнес-логики (Business Logic Layer):** Сосредоточен в контроллере **LibrarianController**. Здесь происходит валидация входных данных, проверка бизнес-правил (лимиты выдачи, наличие задолженностей) и управление транзакциями.
3. **Уровень доступа к данным (Data Access Layer):** Реализован с использованием ORM **Entity Framework Core**. Особенностью реализации является использование **прямых SQL-запросов** (**Database.SqlQuery, ExecuteSqlAsync**) для выполнения всех операций с базой данных, что обеспечивает максимальную производительность и точный контроль над выполняемыми запросами.

4. Уровень хранения данных (Database Layer): Реляционная СУБД **PostgreSQL**, хранящая информацию о книгах, читателях, выдачах и штрафах.

Взаимодействие с БД осуществляется через контекст данных `LibraryDbContext`. Для маппинга (сопоставления) результатов SQL-запросов с объектами `C#` используются специальные DTO-классы (`ReaderInfoDTO`, `ReaderStatusCheck` и др.).

3.2 Разработка программных средств ведения данных

Основным программным модулем является класс `LibrarianController`. Реализованные методы делятся на три группы: транзакционные операции, аналитические отчеты и вспомогательные методы.

Реализация процесса выдачи книги

Метод `IssueBook` реализует сложный алгоритм проверки возможности выдачи. Перед созданием записи в БД выполняется SQL-запрос, проверяющий три условия блокировки:

1. Наличие статуса "Заблокирован" у читателя.
2. Наличие просроченных книг на руках.
3. Превышение лимита активных выдач (максимум 3).

Фрагмент программного кода проверки (SQL-запрос):

```
var checkQuery = @"
SELECT
    (r.reader_statusid = 2) AS ""IsBlocked"",
    EXISTS (
        SELECT 1 FROM issue i_sub
        JOIN issue_item ii_sub ON i_sub.id = ii_sub.issueid
        WHERE i_sub.readerid = r.id
```

```

        AND ii_sub.actual_return_date IS NULL
        AND i_sub.planned_return_date < CURRENT_DATE
    ) AS ""HasOverdue"",
    (SELECT COUNT(*) FROM issue WHERE readerid = r.id AND issue_statusid =
1) AS ""ActiveIssuesCount""
FROM reader r
WHERE r.id = {0}";

```

Для обеспечения целостности данных операции вставки записи о выдаче (INSERT INTO issue) и уменьшения количества книг (UPDATE book) выполняются внутри транзакции BeginTransactionAsync.

Реализация процесса возврата книги

Метод ReturnBook автоматизирует прием книги. Ключевой особенностью является автоматический расчет штрафов и изменение статуса книги в зависимости от её состояния (isDamaged).

Если книга повреждена, система списывает её и начисляет штраф. Если книга возвращена с просрочкой, также фиксируется штраф. Фрагмент реализации логики возврата:

```

// Фиксация возврата
await _context.Database.ExecuteSqlAsync($"@
UPDATE issue_item
SET actual_return_date = {today},
    issue_item_statusid = 2
WHERE id = {issueItemId}");

// Условное начисление штрафа за порчу
if (isDamaged)
{
    await _context.Database.ExecuteSqlAsync($"@
INSERT INTO fine (readerid, issue_itemid, fine_reasonid, fine_statusid, amount,
imposition_date)
VALUES ({info.ReaderId}, {issueItemId}, 2, 1, 1500.00, {today});
}

```

Реализация аналитических функций

Для анализа работы библиотеки реализованы методы GetPopularGenres и GetOverdueStats, использующие агрегатные функции SQL (COUNT, GROUP BY) для формирования статистики по жанрам и динамике должников.

Тестирование программных средств ведения данных

Тестирование разработанного программного обеспечения проводилось методом «черного ящика» с использованием интерфейса Swagger UI. Основной целью тестирования была проверка корректности реализации бизнес-логики, обработки граничных условий и валидации входных данных.

Для проведения испытаний использовалась тестовая база данных, предварительно заполненная данными о читателях, книгах и сотрудниках.

В таблице 11 приведены результаты выполнения контрольных тестовых сценариев.

№	Название теста / Сценарий	Входные данные	Ожидаемый результат	Фактический результат (Ответ API)	Статус
1	Попытка выдачи книги заблокированному читателю	readerId: 10001 (статус «Заблокирован») bookId: 4 librarianId: 2	Отказ в выдаче с указанием причины блокировки.	400 Bad Request "ОТКАЗ: Читатель заблокирован."	Успешно
2	Попытка выдачи книги читателю с просрочкой	readerId: 5 (имеет просроченные книги) bookId: 4 librarianId: 2	Отказ в выдаче из-за наличия долгов.	400 Bad Request "ОТКАЗ: У читателя есть просроченные книги."	Успешно
3	Успешная выдача книги	readerId: 122 (нет долгов) bookId: 4 (есть в наличии) librarianId: 2	Запись создана, количество книг уменьшено. Код 200.	200 OK { "message": "Книга успешно выдана", "issueId": 501 }	Успешно
4	Возврат книги без повреждений (в срок)	issueItemId: 40002 (из теста №3) isDamaged: false	Книга принята, статус изменен на «Доступна».	200 OK { "message": "Книга успешно возвращена." }	Успешно
5	Возврат книги с повреждением	issueItemId: 40003 isDamaged: true	Книга списана, начислен штраф за порчу.	200 OK { "message": "Книга принята с пометкой 'Повреждена'." }	Успешно
6	Попытка повторного возврата книги	issueItemId: 40002 (уже возвращена) isDamaged: false	Сообщение об ошибке, что книга уже сдана.	400 Bad Request "Эта книга уже возвращена."	Успешно

Окончание таблицы 11

№	Название теста / Сценарий	Входные данные	Ожидаемый результат	Фактический результат (Ответ API)	Статус
7	Получение отчета по популярным жанрам	Запрос GET /popular-genres	Список жанров с кол-вом выдач, сортировка по убыванию.	200 OK [{ "genreName": "Фантастика", "borrowCount": 15 }, ...]	Успешно
8	Получение статистики по должникам	Запрос GET /overdue-stats	Данные сгруппированы по годам и месяцам.	200 OK [{ "year": 2024, "month": 5, "debtorsCount": 3 }, ...]	Успешно

Результаты тестов предоставлены на рисунках 10-17.

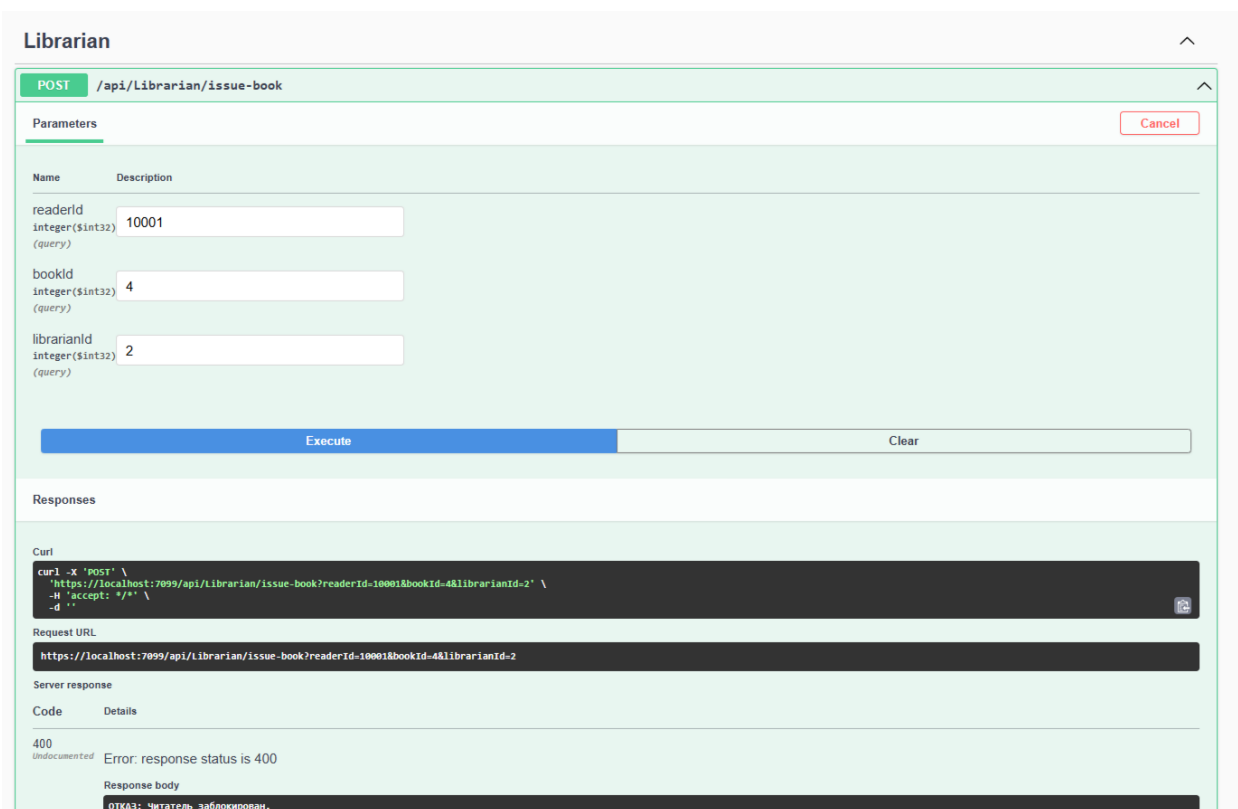


Рисунок 10 – Результат работы теста «Попытка выдачи книги заблокированному читателю»

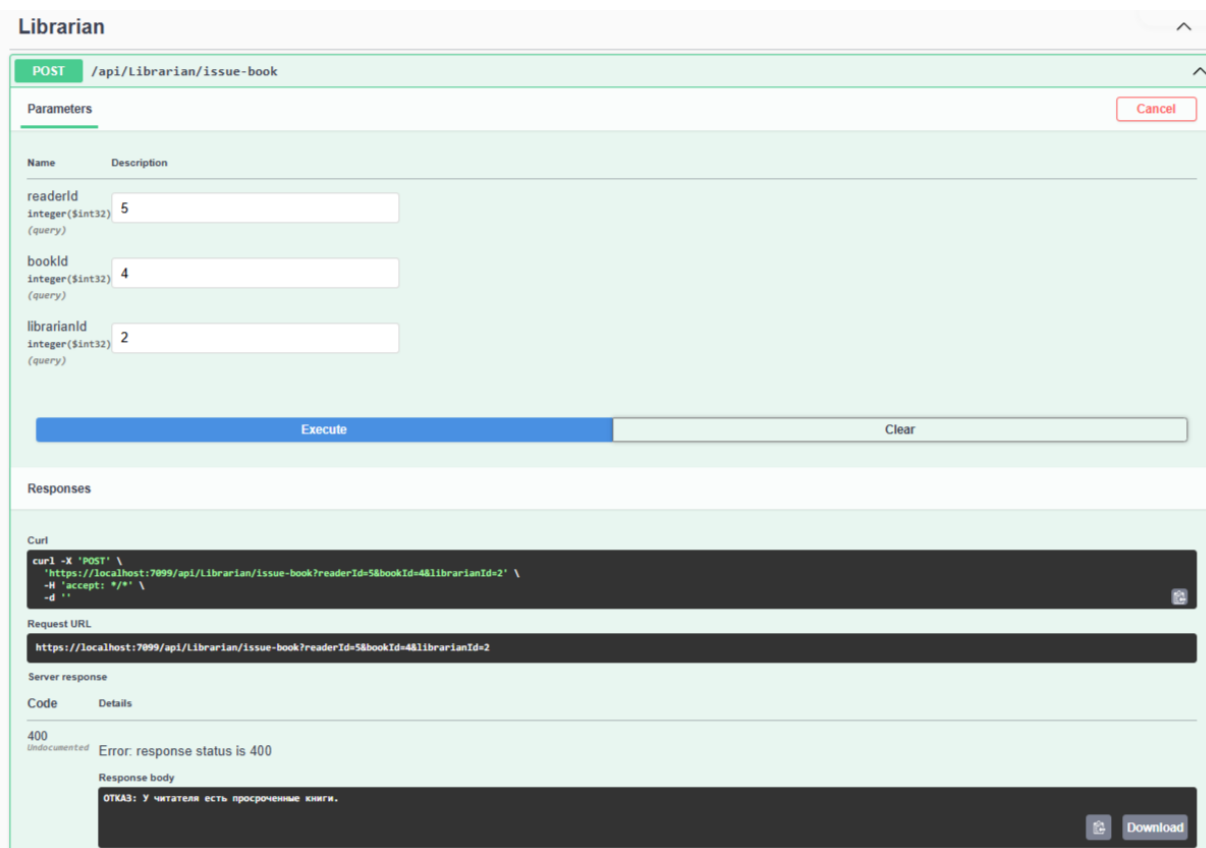


Рисунок 11 – Результат работы теста «Попытка выдачи книги читателю с просрочкой»

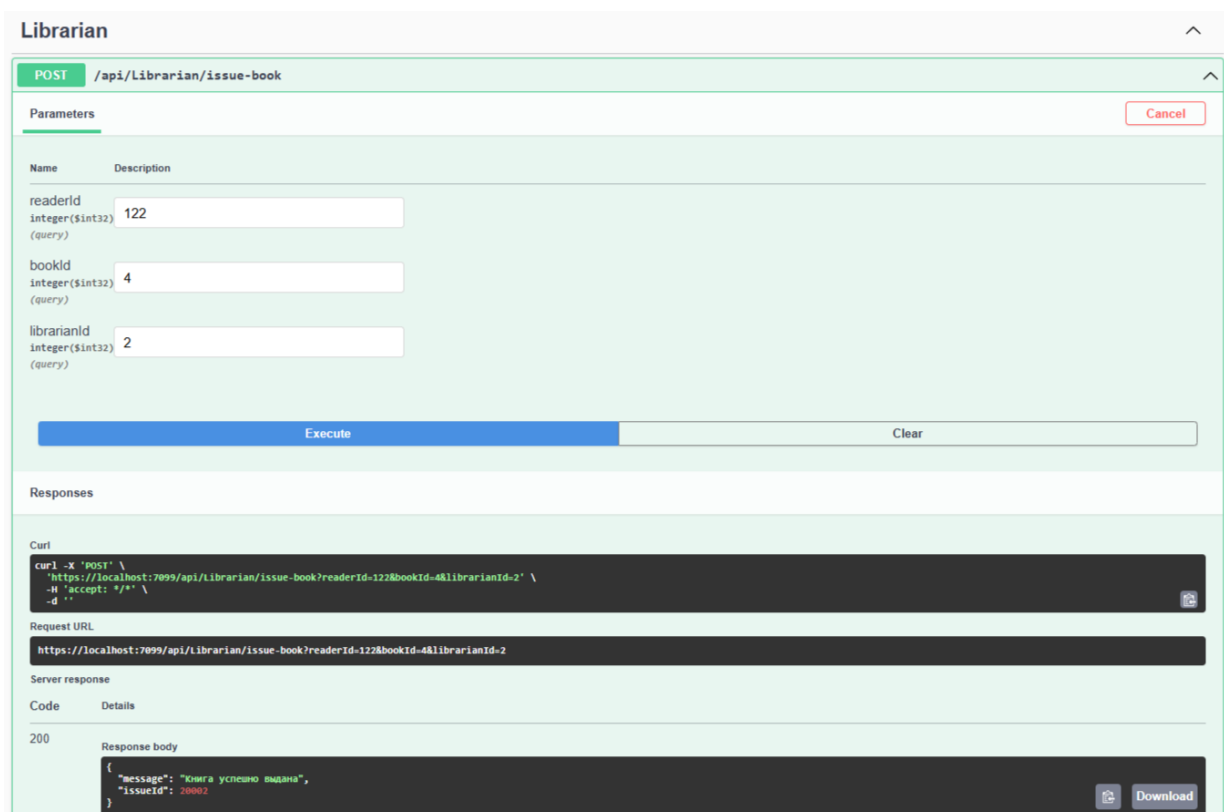


Рисунок 12 – Результат работы теста «Успешная выдача книги»

Librarian

POST /api/Librarian/issue-book

POST /api/Librarian/return-book

Parameters Cancel

Name	Description
issueItemId integer(int32) (query)	40002
isDamaged boolean (query)	false

Execute **Clear**

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:7099/api/Librarian/return-book?issueItemId=40001&isDamaged=false' \
  -H 'accept: */*' \
  -d ''
```

Request URL

https://localhost:7099/api/Librarian/return-book?issueItemId=40001&isDamaged=false

Server response

Code	Details
200	Response body <pre>{ "message": "Книга успешно возвращена." }</pre> Download

Рисунок 13 – Результат работы теста «Возврат книги без повреждений (в срок)»

POST /api/Librarian/return-book

Parameters Cancel

Name	Description
issueItemId integer(int32) (query)	40003
isDamaged boolean (query)	true

Execute **Clear**

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:7099/api/Librarian/return-book?issueItemId=40003&isDamaged=true' \
  -H 'accept: */*' \
  -d ''
```

Request URL

https://localhost:7099/api/Librarian/return-book?issueItemId=40003&isDamaged=true

Server response

Code	Details
200	Response body <pre>{ "message": "Книга принята с пометкой 'повреждена'." }</pre> Download

Рисунок 14 – Результат работы теста «Возврат книги с повреждением»

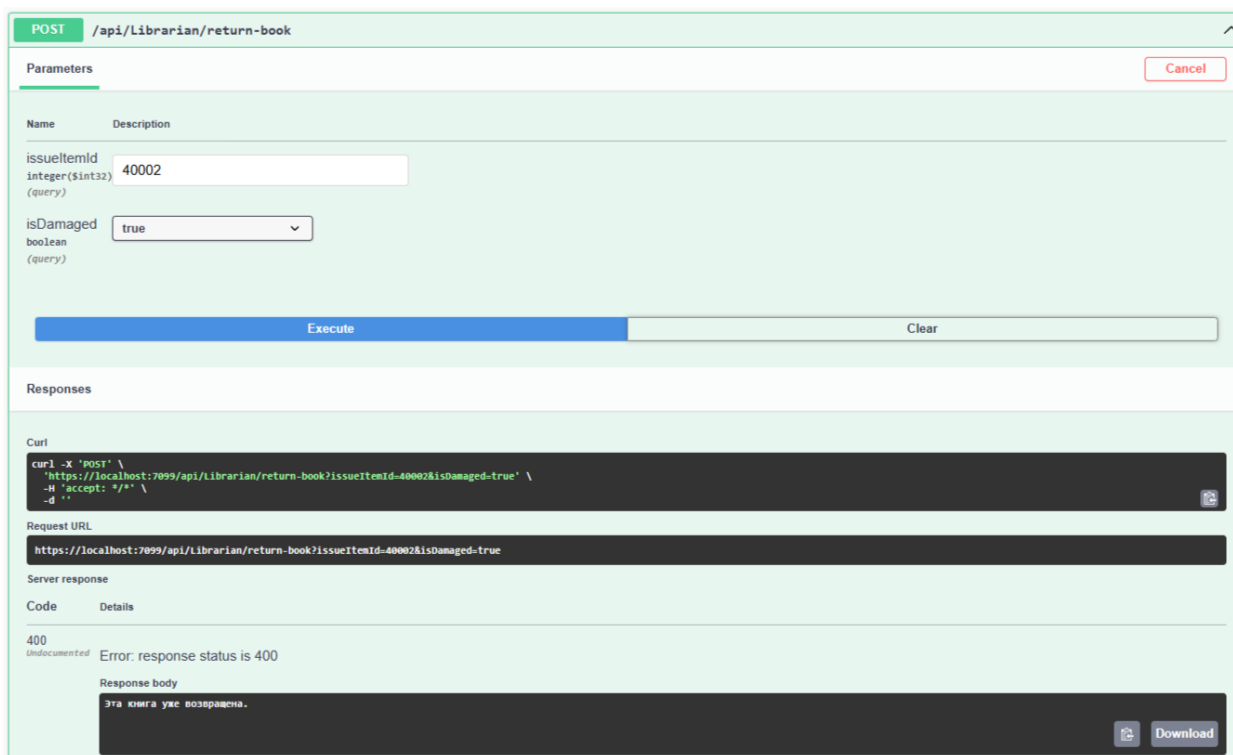


Рисунок 15 – Результат работы теста «Возврат книги с повреждением»

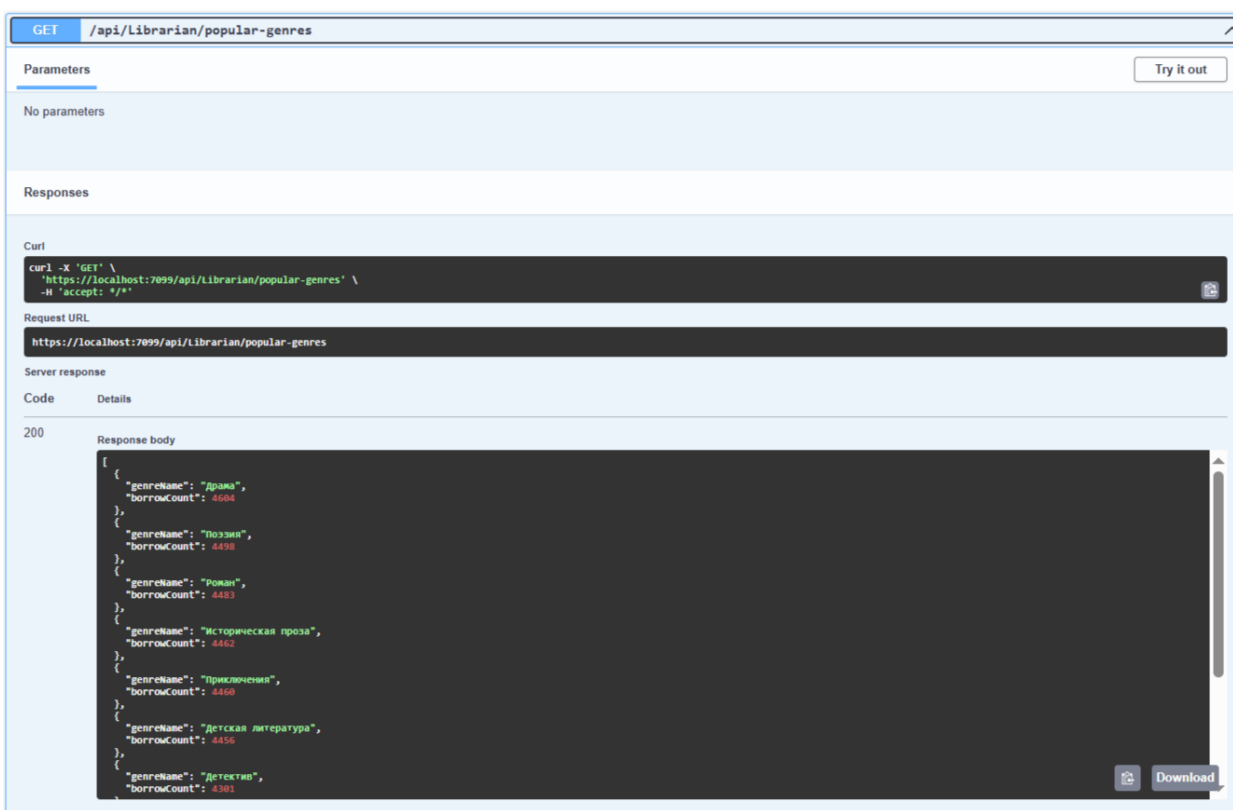


Рисунок 16 – Результат работы теста «Получение отчета по популярным жанрам»

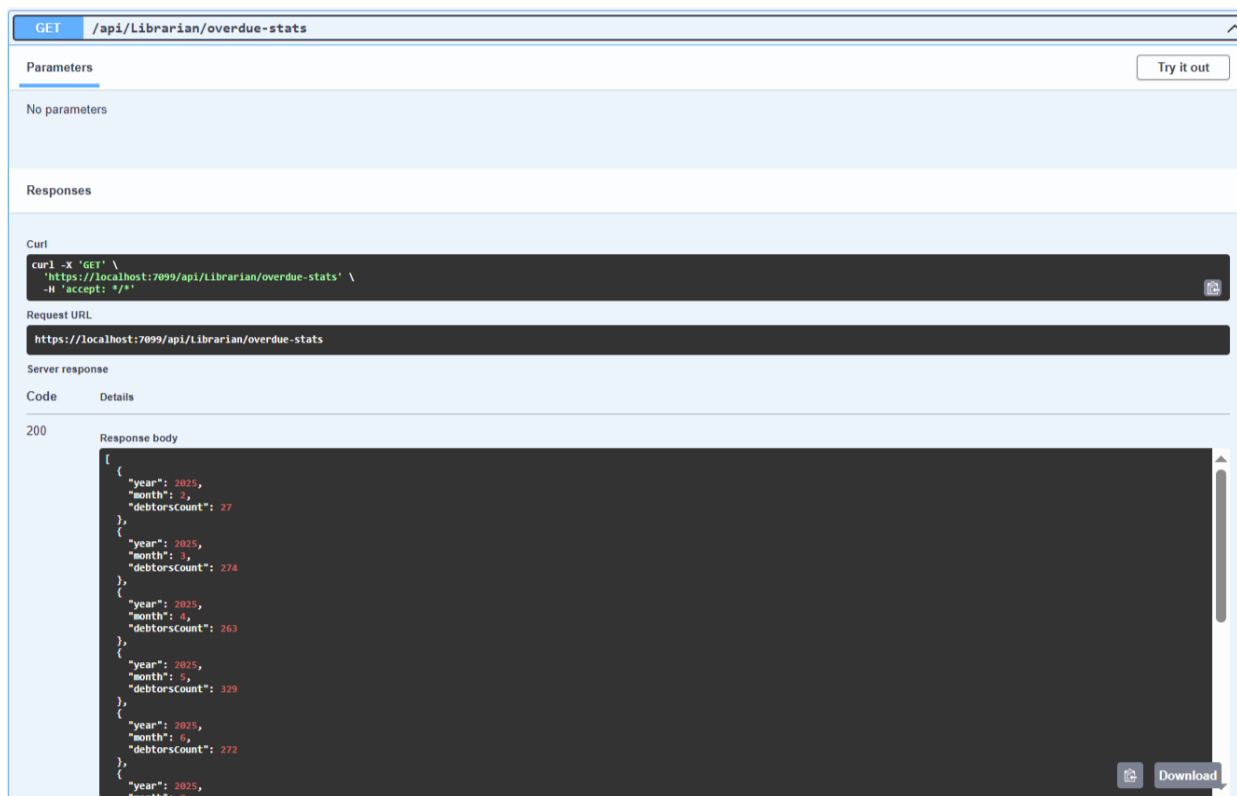


Рисунок 17 – Результат работы теста «Получение статистики по должникам»

Все проведенные тесты завершились успешно. Фактические результаты полностью совпадают с ожидаемыми. Система корректно обрабатывает бизнес-правила (блокировки, лимиты), правильно выполняет транзакции в базе данных PostgreSQL и возвращает понятные сообщения об ошибках при некорректных действиях пользователя. Программные средства готовы к эксплуатации.

Заключение

В ходе выполнения работы были решены следующие задачи:

1. Проведен анализ предметной области «Сеть городских библиотек», описана структурная организация и документооборот.
2. Спроектирована база данных: разработаны концептуальная, логическая и физическая модели.
3. База данных развернута на СУБД PostgreSQL и наполнена тестовыми данными.
4. Разработаны и протестированы программные средства (SQL-скрипты и индексы) для ведения данных библиотекаря, обеспечивающие выполнение основных бизнес-процессов.

Список использованных источников

1. Режим Информационные системы: определение и методологии создания. доступа: <https://otus.ru/journal/informacionnye-sistemy-opredelenie-i-metodologii-sozdaniya/> (дата обращения 24.11.2025)
2. Е.А. Кольчугина. Базы данных. Методические указания к лабораторным работам. / Е.А. Кольчугина, С В. Самуйлов, С.В. Шибанов. Базы данных. Методические указания к лабораторным работам – Пенза: издательство ПГУ, 2000. – 31 с.
3. Концептуальная модель. Режим доступа: <https://mlkv.ru/sample-page/материалы-по-базам-данных/концептуальная-модель/> (дата обращения 24.11.2025)
4. Логическая модель базы данных. Режим доступа: https://studopedia.ru/6_1895_logicheskaya-model-bazi-dannih.html (дата обращения 24.11.2025)
5. Домбровская Генриэтта, Новиков Борис, Бейликова Анна. Оптимизация запросов PostgreSQL -ДМК-Пресс, 2022 г.

Приложение А

Спецификации концептуальной схемы базы данных

Таблица А1 – Спецификации сущностей концептуальной схемы данных

№ п/п	Наименование	Описание	Категория	Количество экземпляров	Режим обновления
1	Читатель	Пользователи библиотеки	Стержневая	~100	10% в месяц
2	Книга	Издания в каталоге	Стержневая	~20	5% в месяц
3	Экземпляр_Книги	Физические копии	Ассоциативная	~100	7% в месяц
4	Выдача	Выдача книг	Ассоциативная	~10 активных	20% в день
5	Заявка_На_Закупку	Запросы на пополнение	Ассоциативная	~120 в год	10% в месяц
6	Поставщик	Организации-поставщики	Стержневая	~10	1 раз в квартал
7	Поставка	Поступления книг	Ассоциативная	~120 в год	10% в месяц
8	Штраф	Начисления за нарушения	Ассоциативная	~5 активных	20% в неделю
9	Библиотекарь	Сотрудник библиотеки	Стержневая	5	20% в год

Таблица А2 – Спецификации связей между сущностями концептуальной схемы данных

№ п/п	Сущность 1	Сущность 2	Семантика связи	Мощность	Обязательность	Ограничения
1	Читатель	Выдача	Оформляет	1:M	Обязательная для Выдачи	Читатель должен быть активен (Статус = "Активен")
2	Книга	Экземпляр_Книги	Имеет	1:M	Обязательная для Экземпляра	-
3	Книга	Выдача	Заказывается в	1:M	Обязательная для Выдачи	Книга должна иметь Доступное_Количество > 0
4	Книга	Заявка_На_Закупку	Запрашивается в	1:M	Обязательная для Заявки	-
5	Книга	Поставка	Поставляется в	1:M	Обязательная для Поставки	-
6	Поставщик	Поставка	Осуществляет	1:M	Обязательная для Поставки	-
7	Читатель	Штраф	Начисляются	1:M	Обязательная для Штрафа	-
8	Выдача	Штраф	Связан с	1:1	Необязательная	Штраф начисляется только для выдачи с просрочкой
9	Библиотекарь	Выдача	Оформляет	1:M	Обязательная для Выдачи	-

Таблица А3.1 - Спецификации атрибутов сущности «Читатель»

№ п/п	Наименование	Описание	Тип данных	Размер (в символах)	Роль	Обязательный	Ограничения значений	Значение по умолчанию
1	ID_Читателя	Уникальный идентификатор читателя	Integer	10	PK	Да	Auto-increment, > 0	Auto-increment
2	Фамилия	Полная фамилия читателя	Varchar	255		Да	Только буквы	-
3	Имя	Полное имя читателя	Varchar	255		Да	Только буквы	-
4	Отчество	Полное отчество читателя (при наличии)	Varchar	255		Нет	Только буквы	-
5	Телефон	Номер телефона	Varchar	20	UQ	Да	Только цифры, +, -	-
6	Пароль	Хэш пароля	Varchar	20		Да	от 6 до 20 символов	-
7	Дата_Регистрации	Дата регистрации в системе	Date			Да	≤ текущей даты	CURRENT_DATE
8	Статус	Статус учетной записи	Varchar	255		Да	Активен/Не активен/Удалён	Не активен

Таблица А3.2 - Спецификации атрибутов сущности «Книга»

№ п/п	Наименование	Описание	Тип данных	Размер (в символах)	Роль	Обязательный	Ограничения значений	Значение по умолчанию
1	ID_Книги	Уникальный идентификато р книги	Integer	10	РК	Да	Auto- increment, > 0	Auto-increment
2	Название	Название книги	Varchar	255		Да	-	-
3	Автор	Автор(ы) книги	Varchar	150		Да	-	-
4	Жанр	Жанр литературы	Varchar	50		Да	-	-
5	Общее_Количе ство	Всего экземпляров	Integer	4		Да	≥ 0	0
6	Доступное_Кол ичество	Доступно для выдачи	Integer	4		Да	≥ 0 , $\leq$ Общее_Колич ество	0

Таблица А3.3 - Спецификации атрибутов сущности «Экземпляр_Книги»

№ п/п	Наименование	Описание	Тип данных	Размер (в символах)	Роль	Обязательный	Ограничения значений	Значение по умолчанию
1	ID_Экземпляра	Уникальный идентификатор экземпляра книги	Integer	10	PK	Да	Auto-increment, > 0	Auto-increment
2	Книга	Ссылка на книгу	Integer	10		Да	Существующий ID_Книги	-
3	Инвентарный_Номер	Уникальный инвентарный номер	Varchar	255	UQ	Да	-	3
4	Статус	Текущий статус экземпляра	Varchar	255		Да	Доступна/Выдана/Списана	Доступна
5	Дата_Поступления	Дата поступления в фонд	Date			Да	≤ текущей даты	CURRENT_DATE

Таблица А3.4 - Спецификации атрибутов сущности «Выдачи»

№ п/п	Наименование	Описание	Тип данных	Размер (в символах)	Роль	Обязательный	Ограничения значений	Значение по умолчанию
1	ID_Выдачи	Уникальный идентификатор выдачи	Integer	10	PK	Да	Auto-increment, > 0	Auto-increment
2	Читатель	Ссылка на читателя	Integer	10		Да	Существующий ID_Читателя	-
3	Книга	Ссылка на книгу	Integer	10		Да	Существующий ID_Книги	-
4	Библиотекарь	Ссылка на библиотекаря	Integer	10		Да	Существующий ID_Библиотекаря	
5	Дата_Запроса	Дата создания запроса на выдачу	Date			Да	≤ текущей даты	CURRENT_DATE
6	Статус	Статус заказа	Varchar	255		Да	Ожидание/Готов/Выдан/Отменен	Ожидание
7	Время_Выдачи	Фактическое время выдачи	Time			Нет	-	NULL
8	Планируемая_Дата_Возврата	Плановая дата возврата	Date			Нет	> Дата_Выдачи	NULL
9	Реальная дата возврата	Дата, когда книгу вернули	Date			Нет	> Дата_Выдачи	NULL

Таблица А3.5 - Спецификации атрибутов сущности «Заявка_На_Закупку»

№ п/п	Наименование	Описание	Тип данных	Размер (в символах)	Роль	Обязательный	Ограничения значений	Значение по умолчанию
1	ID_Заявки	Уникальный идентификато р заявки	Integer	10	PK	Да	Auto- increment, > 0	Auto-increment
2	Книга	Ссылка на книгу	Integer	10		Да	Существующ ий ID_Книги	-
3	Количество	Запрашиваем ое количество	Integer	10		Да	> 0, ≤ 100	1
4	Статус	Статус заявки	Varchar	255		Да	Новая/В работе/Выпол нена/Отклоне на	Новая
5	Дата_Создания	Дата создания заявки	Date			Да	≤ текущей даты	CURRENT_DA TE

Таблица А3.6 - Спецификации атрибутов сущности «Поставщик»

№ п/п	Наименование	Описание	Тип данных	Размер (в символах)	Роль	Обязательный	Ограничения значений	Значение по умолчанию
1	ID_Поставщик а	Уникальный идентификато р поставщика	Integer	10	РК	Да	Auto- increment, > 0	Auto-increment
2	Название	Наименовани е организации	Varchar	200		Да	-	-
3	Адрес	Юридический адрес	Varchar	300		Да	-	-
4	Телефон	Контактный телефон	Varchar	20		Да	Только цифры, +, -	-
5	Рейтинг	Рейтинг надёжности	Integer	1		Да	1-5	3

Таблица А3.7 - Спецификации атрибутов сущности «Поставка»

№ п/п	Наименование	Описание	Тип данных	Размер (в символах)	Роль	Обязательный	Ограничения значений	Значение по умолчанию
1	ID_Поставки	Уникальный идентификатор поставки	Integer	10	PK	Да	Auto-increment, > 0	Auto-increment
2	Поставщик	Ссылка на поставщика	Integer	10	FK	Да	Существующий ID_Поставщика	-
3	Книга	Ссылка на книгу	Integer	10	FK	Да	Существующий ID_Книги	-
4	Количество	Количество экземпляров	Integer	4		Да	> 0	1
5	Цена_За_Единицу	Цена за один экземпляр	Decimal	(10,2)		Да	> 0	0.00
6	Дата Заявки	Дата, когда была подана заявка	Date			Да	≤ текущей даты	CURRENT_DATE
7	Дата_Поставки	Фактическая дата поставки	Date			Да	≤ текущей даты	CURRENT_DATE
8	Статус	Статус поставки	Varchar	20		Да	Ожидается/Получена/Проверена	Ожидается

Таблица А3.8 - Спецификации атрибутов сущности «Штраф»

№ п/п	Наименование	Описание	Тип данных	Размер (в символах)	Роль	Обязательный	Ограничения значений	Значение по умолчанию
1	ID_Штрафа	Уникальный идентификатор штрафа	Integer	10	PK	Да	Auto-increment, > 0	Auto-increment
2	Читатель	Ссылка на читателя	Integer	10		Да	Существующий ID Читателя	-
3	Выдача	Ссылка на Выдачу	Integer	10		Да	Существующий ID_Выдачи	-
4	Причина	Причина начисления штрафа	Varchar	255		Да	Просрочка/Порча	-
5	Сумма	Сумма штрафа	Decimal	(10,2)		Да	> 0	0.00
6	Статус	Статус штрафа	Varchar	255		Да	Неоплачен/Оплачен	Неоплачен
	Дата_Начисления	Дата начисления штрафа	Date			Да	≤ текущей даты	CURRENT_DATE
8	Дата_Оплаты	Дата оплаты штрафа	Date			Нет	≥ Дата_Начисления	NULL

Таблица А3.9 - Спецификации атрибутов сущности «Библиотекарь»

№ п/п	Наименование	Описание	Тип данных	Размер (в символах)	Роль	Обязательный	Ограничения значений	Значение по умолчанию
1	ID_Библиотекаря	Уникальный идентификатор библиотекаря	Integer	10	РК	Да	Auto-increment, > 0	Auto-increment
2	Фамилия	Полная фамилия библиотекаря	Varchar	255		Да	Только буквы	-
3	Имя	Полное имя библиотекаря	Varchar	255		Да	Только буквы	-
4	Отчество	Полное отчество библиотекаря (при наличии)	Varchar	255		Нет	Только буквы	-
7	Дата_трудоустройства	Дата приёма на работу	Date			Да	≤ текущей даты	CURRENT_DATE

Приложение Б

Спецификации логической схемы базы данных

Таблица Б1 – Спецификация отношений

Имя отношения	Тип	Описание
Библиотека	Независимое	Содержит информацию о филиалах сети городских библиотек.
Читатель	Независимое	Содержит информацию о зарегистрированных пользователях библиотеки с привязкой к библиотеке регистрации.
Книга	Независимое	Содержит информацию об изданиях в каталоге библиотеки.
Автор	Независимое	Справочник авторов литературных произведений.
Жанр	Независимое	Справочник жанров литературных произведений.
Книга_Автор	Зависимое (ассоциативное)	Разрешение связи "многие-ко-многим" между книгами и авторами.
Экземпляр_Книги	Зависимое	Содержит информацию о конкретных физических копиях книг.
Сотрудник	Независимое	Содержит информацию о всех сотрудниках библиотечной сети с привязкой к должности и библиотеке.
Поставщик	Независимое	Содержит информацию об организациях-поставщиках литературы.
Выдача	Независимое	Учет операций выдачи книг читателям (заголовок заказа).
Позиция_Выдачи	Зависимое	Учет конкретных книг в составе выдачи.
Заявка_На_Закупку	Зависимое	Учет заявок на пополнение библиотечного фонда.
Позиция_Заявки_На_Закупку	Зависимое	Учет конкретных книг в составе заявки на закупку.
Поставка	Независимое	Учет партий книг, поступивших от поставщиков (заголовок поставки).
Позиция_Поставки	Зависимое	Учет конкретных книг в составе поставки.
Штраф	Зависимое	Учет начисленных штрафов за нарушение правил библиотеки.
Должность	Справочное	Справочник должностей сотрудников (Библиотекарь, менеджер по поставкам, работник книжного фонда).
Заявка_Менеджеру	Зависимое	Учет внутренних заявок и поручений между сотрудниками.
Статус_Читателя	Справочное	Справочник возможных статусов читательского билета (Активен, Заблокирован).
Статус_Выдачи	Справочное	Справочник статусов операции выдачи (Активна, Завершена, Просрочена, Отменена).
Статус_Позиции_В_ыдачи	Справочное	Справочник статусов конкретной книги в выдаче (Выдана, Возвращена).
Статус_Экземпляра_Книги	Справочное	Справочник статусов физического экземпляра книги (Доступен, Выдан, Списана).
Статус_Заявки_На_Закупку	Справочное	Справочник статусов заявки на закупку (Новая, Одобрена, Выполнена, Отклонена).
Статус_Позиции_За_явки_На_Закупку	Справочное	Справочник статусов позиции заявки на конкретную книгу (Новая, Одобрена, Выполнена, Отклонена)

Окончание таблицы Б1

Имя отношения	Тип	Описание
Статус_Поставки	Справочное	Справочник статусов поставки (Ожидается, Получена, Проверена).
Статус_Позиции_Поставки	Справочное	Справочник статусов конкретной поставки партии книги (Получена, Проверена)
Статус_Заявки_Менеджеру	Справочное	Справочник статусов внутренней заявки (Новая, Выполнена, Отменена).
Статус_Штрафа	Справочное	Справочник статусов штрафа (Активен, Оплачен)
Причина_Штрафа	Справочное	Справочник причин выписывания штрафа (Просрочка, Порча)

Таблица Б2 - Спецификация связей между отношениями

Отношения связи	Обязательность	Мощность	Смысл связи		Тип
			Parent to Child	Child to Parent	
Библиотека → Сотрудник	No Nulls	One-to-Many	В ней работают	Работает в библиотеке	Non-identifying
Должность → Сотрудник	No Nulls	One-to-Many	Занимают должность	Должность занимают	Non-identifying
Библиотека → Читатель	No Nulls	One-to-Many	В ней зарегистрированы	Зарегистрирован в библиотеке	Non-identifying
Читатель → Выдача	No Nulls	One-to-Many	Оформляет заказы	Оформлен читателем	Non-identifying
Сотрудник → Выдача	No Nulls	One-to-Many	Оформляет выдачи	Оформлена сотрудником	Non-identifying
Выдача → Позиция_Выдачи	No Nulls	One-to-Many	Содержит позиции	Входит в выдачу	Identifying
Позиция_Выдачи → Книга	No Nulls	Many-to-One	Относится к книге	Выдана в позициях	Non-identifying
Книга → Книга_Автор	No Nulls	One-to-Many	Имеет авторов	Написана автором	Identifying
Автор → Книга_Автор	No Nulls	One-to-Many	Написал книги	Автор книги	Identifying
Жанр → Книга	No Nulls	One-to-Many	Содержит книги	Принадлежит жанру	Non-identifying
Книга → Экземпляр_Книги	No Nulls	One-to-Many	Имеет экземпляры	Является копией	Non-identifying
Поставщик → Поставка	No Nulls	One-to-Many	Осуществляет поставки	Поставлен поставщиком	Non-identifying
Поставка → Позиция_Поставки	No Nulls	One-to-Many	Содержит позиции	Входит в поставку	Identifying
Заявка_На_Закупку → Позиция_Заявки_На_Закупку	No Nulls	One-to-Many	Содержит позиции	Входит в заявку	Identifying

Отношения связи	Обязательность	Мощность	Смысл связи		Тип
			Parent to Child	Child to Parent	
Позиция_Поставки → Позиция_Заявки_На_Закупку	No Nulls	One-to-One	Содержит позицию Заявки	Входит в позицию поставки	Identifying

Окончание таблицы Б2

Отношения связи	Обязательность	Мощность	Смысл связи		Тип
			Parent to Child	Child to Parent	
Читатель → Штраф	No Nulls	One-to-Many	Имеет штрафы	Начислен читателю	Non-identifying
Позиция_Выдачи → Штраф	No Nulls	One-to-Zero-or-One	Может повлечь штраф	Начислен за позицию	Non-identifying
Книга → Позиция_Заявки_На_Закупку	No Nulls	One-to-Many	Запрашивается к закупке	Заявлена на закупку	Non-identifying
Книга → Позиция_Поставки	No Nulls	One-to-Many	Поставляется в составе	Поставлена как книга	Non-identifying
Сотрудник → Заявка_Менеджеру	No Nulls	One-to-Many	Назначен исполнителем	Назначена менеджеру	Non-identifying
Статус_Читателя → Читатель	No Nulls	One-to-Many	Определяет статус	Имеет статус	Non-identifying
Статус_Выдачи → Выдача	No Nulls	One-to-Many	Определяет статус	Имеет статус	Non-identifying
Статус_Позиции_Выдачи → Позиция_Выдачи	No Nulls	One-to-Many	Определяет статус	Имеет статус	Non-identifying
Статус_Экземпляра_Книги → Экземпляр_Книги	No Nulls	One-to-Many	Определяет статус	Имеет статус	Non-identifying
Статус_Заявки_На_Закупку → Заявка_На_Закупку	No Nulls	One-to-Many	Определяет статус	Имеет статус	Non-identifying
Статус_Поставки → Поставка	No Nulls	One-to-Many	Определяет статус	Имеет статус	Non-identifying
Статус_Позиции_Поставки → Позиция_Поставки	No Nulls	One-to-Many	Определяет статус	Имеет статус	Non-identifying
Статус_Заявки_Менеджеру → Заявка_Менеджеру	No Nulls	One-to-Many	Определяет статус	Имеет статус	Non-identifying
Статус_Штрафа → Штраф	No Nulls	One-to-Many	Определяет статус	Имеет статус	Non-identifying
Статус_Позиции_Заявки_На_Закупку → Позиция_Заявки_На_Закупку	No Nulls	One-to-Many	Определяет статус	Имеет статус	Non-identifying

Отношения связи	Обязательность	Мощность	Смысл связи		Тип
			Parent to Child	Child to Parent	
Причина_Штрафа → Штраф	No Nulls	One-to-Many	Определяет причину	Имеет причину	Non-identifying
Поставщик → Заявка_На_Закупку	No Nulls	One-to-Many	Предлагается для закупки	Заявлен поставщик	Non-identifying
Книга → Заявка_Менеджеру	No Nulls	One-to-Many	Имеет связанные заявки	Относится к книге	Non-identifying

Таблица Б3.1 – Спецификации атрибутов отношения «Библиотека»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER	Yes	Уникальный идентификатор библиотеки	Yes	No
Название	VARCHAR(255)	Yes	Название библиотеки-филиала	No	No
Улица	VARCHAR(100)	Yes	Улица местонахождения библиотеки	No	No
Дом	VARCHAR(10)	Yes	Дом местонахождения библиотеки	No	No
Телефон	VARCHAR(20)	Yes	Контактный телефон библиотеки	No	No

Таблица Б3.2 – Спецификации атрибутов отношения «Читатель»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER	Yes	Уникальный идентификатор читателя	Yes	No
Фамилия	VARCHAR(100)	Yes	Фамилия читателя	No	No
Имя	VARCHAR(100)	Yes	Имя читателя	No	No
Отчество	VARCHAR(100)	No	Отчество читателя	No	No
Телефон	VARCHAR(20)	Yes	Контактный телефон	No	No
Пароль	VARCHAR(255)	Yes	Хэш пароля для входа в систему	No	No
Дата_Регистрации	DATE	Yes	Дата регистрации в библиотеке	No	No
ID_Статуса	INTEGER	Yes	Идентификатор статуса читателя	No	Yes
ID_Библиотеки_Регистрации	INTEGER	Yes	Идентификатор библиотеки, где зарегистрирован читатель	No	Yes

Таблица Б3.3 – Спецификации атрибутов отношения «Книга»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER	Yes	Уникальный идентификатор книги	Yes	No
Название	VARCHAR(255)	Yes	Название	No	No

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
			литературного произведения		
ID_Жанра	INTEGER	Yes	Идентификатор жанра произведения	No	Yes
Общее_Количество	INTEGER	Yes	Общее количество экземпляров в фонде	No	No
Доступное_Количество	INTEGER	Yes	Количество доступных для выдачи экземпляров	No	No

Таблица Б3.4– Спецификации атрибутов отношения «Автор»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER	Yes	Уникальный идентификатор автора	Yes	No
Фамилия	VARCHAR(100)	Yes	Фамилия автора	No	No
Имя	VARCHAR(100)	Yes	Имя автора	No	No
Отчество	VARCHAR(100)	No	Отчество автора	No	No

Таблица Б3.5– Спецификации атрибутов отношения «Книга_Автор»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID_Книги	INTEGER	Yes	Идентификатор книги	Yes	Yes
ID_Автора	INTEGER	Yes	Идентификатор автора	Yes	Yes

Таблица Б3.6– Спецификации атрибутов отношения «Жанр»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER	Yes	Уникальный идентификатор жанра	Yes	No
Название	VARCHAR(50)	Yes	Наименование жанра	No	No

Таблица Б3.7– Спецификации атрибутов отношения «Экземпляр_Книги»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER	Yes	Уникальный идентификатор экземпляра	Yes	No
ID_Книги	INTEGER	Yes	Идентификатор книги	No	Yes
Инвентарный_Номер	VARCHAR(50)	Yes	Уникальный инвентарный номер экземпляра	No	No
ID_Статуса	INTEGER	Yes	Идентификатор статуса экземпляра	No	Yes
Дата_Поступления	DATE	Yes	Дата поступления экземпляра в фонд	No	No

Таблица Б3.8– Спецификации атрибутов отношения «Сотрудник»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER	Yes	Уникальный идентификатор сотрудника	Yes	No

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
Фамилия	VARCHAR(100)	Yes	Фамилия сотрудника	No	No
Имя	VARCHAR(100)	Yes	Имя сотрудника	No	No
Отчество	VARCHAR(100)	No	Отчество сотрудника	No	No
ID_Библиотеки	INTEGER	Yes	Идентификатор библиотеки места работы	No	Yes
ID_Должности	INTEGER	Yes	Идентификатор должности сотрудника	No	Yes
Дата_Трудоустройства	DATE	Yes	Дата приема на работу	No	No

Таблица Б3.9– Спецификации атрибутов отношения «Поставщик»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER	Yes	Уникальный идентификатор поставщика	Yes	No
Название	VARCHAR(200)	Yes	Наименование организации-поставщика	No	No
Город	VARCHAR(50)	Yes	Город местонахождения поставщика	No	No
Улица	VARCHAR(100)	Yes	Улица местонахождения поставщика	No	No
Дом	VARCHAR(10)	Yes	Дом местонахождения поставщика	No	No
Телефон	VARCHAR(20)	Yes	Контактный телефон поставщика	No	No
Рейтинг	INTEGER	Yes	Рейтинг надежности поставщика (1-5)	No	No

Таблица Б3.10– Спецификации атрибутов отношения «Выдача»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER	Yes	Уникальный идентификатор выдачи	Yes	No
ID_Читателя	INTEGER	Yes	Идентификатор читателя	No	Yes
ID_Сотрудника	INTEGER	Yes	Идентификатор сотрудника, оформившего выдачу	No	Yes
ID_Статуса	INTEGER	Yes	Идентификатор статуса выдачи	No	Yes
Дата_Запроса	DATE	Yes	Дата оформления выдачи	No	No
Планируемая_Дата_Возврата	DATE	No	Плановая дата возврата книг	No	No

Таблица Б3.11– Спецификации атрибутов отношения «Позиция_Выдачи»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER	Yes	Уникальный идентификатор позиции	Yes	No
ID_Выдачи	INTEGER	Yes	Идентификатор родительской выдачи	No	Yes
ID_Книги	INTEGER	Yes	Идентификатор книги в позиции	No	Yes
ID_Статуса	INTEGER	Yes	Идентификатор статуса позиции выдачи	No	Yes

Окончание таблицы Б3.11

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
Реальная_Дата_Возврата	DATE	No	Фактическая дата возврата книги	No	No

Таблица Б3.12– Спецификации атрибутов отношения «Заявка_На_Закупку»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER	Yes	Уникальный идентификатор заявки	Yes	No
ID_Статуса	INTEGER	Yes	Идентификатор статуса заявки	No	Yes
Дата_Создания	DATE	Yes	Дата создания заявки	No	No
ID_Поставщика	INTEGER	Yes	Идентификатор поставщика	No	Yes

Таблица Б3.13– Спецификации атрибутов отношения

«Позиция_Заявки_На_Закупку»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER	Yes	Уникальный идентификатор заявки	Yes	No
ID_Статуса	INTEGER	Yes	Идентификатор статуса позиции_заявки	No	Yes
ID_Поставщика	INTEGER	Yes	Идентификатор поставщика	No	Yes
ID_Книги	INTEGER	Yes	Идентификатор книги для закупки	No	Yes
Количество	INTEGER	Yes	Запрашиваемое количество экземпляров	No	No

Таблица Б3.14– Спецификации атрибутов отношения «Поставка»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER	Yes	Уникальный	Yes	No

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
			идентификатор поставки		
ID_Поставщика	INTEGER	Yes	Идентификатор поставщика	No	Yes
Дата_Заявки	DATE	Yes	Дата размещения заявки на поставку	No	No
Дата_Поставки	DATE	Yes	Фактическая дата поставки	No	No
ID_Статуса	INTEGER	Yes	Идентификатор статуса поставки	No	Yes
Общая_Стоимость	DECIMAL(12,2)	No	Общая стоимость поставки	No	No

Таблица Б3.15– Спецификации атрибутов отношения «Позиция_Поставки»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER	Yes	Уникальный идентификатор позиции	Yes	No
ID_Поставки	INTEGER	Yes	Идентификатор родительской поставки	No	Yes
ID_Книги	INTEGER	Yes	Идентификатор книги в позиции	No	Yes
Количество	INTEGER	Yes	Количество экземпляров в позиции	No	No
Цена_За_Единицу	DECIMAL(10,2)	Yes	Цена за один экземпляр книги	No	No
ID_Статуса	INTEGER	Yes	Идентификатор статуса позиции поставки	No	Yes
ID_Позиции_Заявки_На_Закупку	INTEGER	Yes	Идентификатор позиции заявки на закупку	No	Yes

Таблица Б3.16– Спецификации атрибутов отношения «Штраф»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER	Yes	Уникальный идентификатор штрафа	Yes	No
ID_Читателя	INTEGER	Yes	Идентификатор читателя	No	Yes
ID_Позиции_Выдачи	INTEGER	Yes	Идентификатор позиции выдачи	No	Yes
ID_Причины	INTEGER	Yes	Идентификатор причины штрафа	No	Yes
Сумма	DECIMAL(10,2)	Yes	Сумма штрафа	No	No

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID_Статуса	INTEGER	Yes	Идентификатор статуса штрафа	No	Yes
Дата_Начисления	DATE	Yes	Дата начисления штрафа	No	No
Дата_Оплаты	DATE	No	Дата оплаты штрафа	No	No

Таблица Б3.17– Спецификации атрибутов отношения «Должность»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER	Yes	Уникальный идентификатор должности	Yes	No
Название	VARCHAR(100)	Yes	Наименование должности	No	No

Таблица Б3.18– Спецификации атрибутов отношения «Заявка_Менеджеру»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER(10)	Yes	Уникальный идентификатор заявки	Yes	No
Дата_Заявки	DATE	Yes	Дата создания заявки	No	No
ID_Сотрудника	INTEGER(10)	Yes	Идентификатор сотрудника, создавшего заявку	No	Yes
ID_Книги	INTEGER(10)	Yes	Идентификатор книги, к которой относится заявка	No	Yes
ID_Статуса	INTEGER(10)	Yes	Идентификатор статуса заявки	No	Yes

Таблица Б3.19– Спецификации атрибутов отношения «Статус_Выдачи»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER	Yes	Уникальный идентификатор статуса	Yes	No
Название	VARCHAR(50)	Yes	Наименование статуса	No	No

Таблица Б3.20– Спецификации атрибутов отношения «Статус_Читателя»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER	Yes	Уникальный идентификатор статуса	Yes	No
Название	VARCHAR(50)	Yes	Наименование статуса	No	No

Таблица Б3.21– Спецификации атрибутов отношения

«Статус_Позиции_Выдачи»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER	Yes	Уникальный идентификатор статуса	Yes	No
Название	VARCHAR(50)	Yes	Наименование статуса	No	No

Таблица Б3.22– Спецификации атрибутов отношения

«Статус_Экземпляра_Книги»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER	Yes	Уникальный идентификатор статуса	Yes	No
Название	VARCHAR(50)	Yes	Наименование статуса	No	No

Таблица Б3.23– Спецификации атрибутов отношения

«Статус_Заявки_На_Закупку»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER	Yes	Уникальный идентификатор статуса	Yes	No
Название	VARCHAR(50)	Yes	Наименование статуса	No	No

Таблица Б3.24– Спецификации атрибутов отношения «Статус_Поставки»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER	Yes	Уникальный идентификатор статуса	Yes	No
Название	VARCHAR(50)	Yes	Наименование статуса	No	No

Таблица Б3.25– Спецификации атрибутов отношения

«Статус_Позиции_Поставки»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER	Yes	Уникальный идентификатор статуса	Yes	No
Название	VARCHAR(50)	Yes	Наименование статуса	No	No

Таблица Б3.26– Спецификации атрибутов отношения

«Статус_Заявки_Менеджеру»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER	Yes	Уникальный идентификатор	Yes	No

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
			статуса		
Название	VARCHAR(50)	Yes	Наименование статуса	No	No

Таблица Б3.27– Спецификации атрибутов отношения «Статус_Штрафа»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER	Yes	Уникальный идентификатор статуса	Yes	No
Название	VARCHAR(50)	Yes	Наименование статуса	No	No

Таблица Б3.28– Спецификации атрибутов отношения «Причина_Штрафа»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER	Yes	Уникальный идентификатор статуса	Yes	No
Название	VARCHAR(50)	Yes	Наименование статуса	No	No

Таблица Б3.29– Спецификации атрибутов отношения

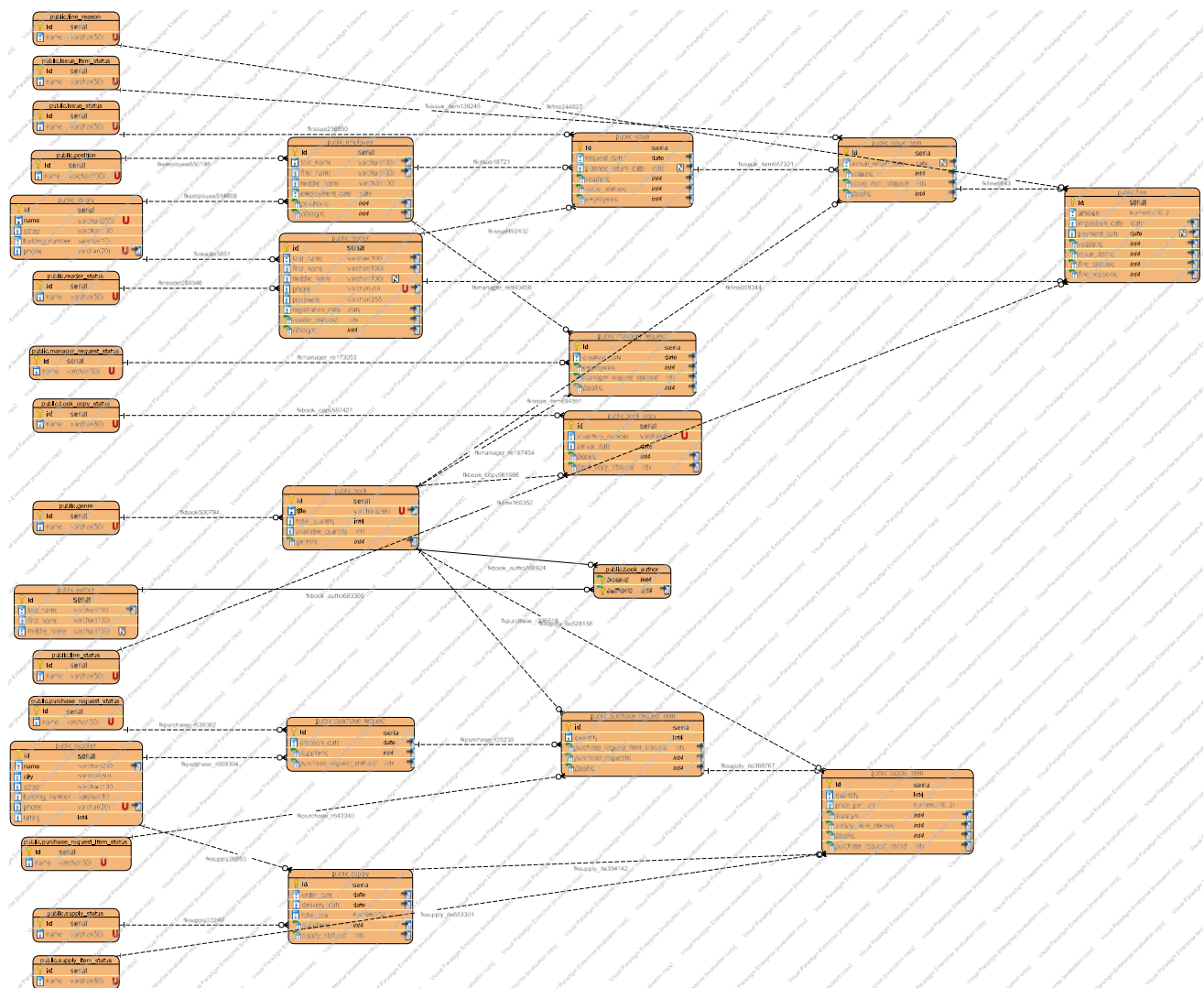
«Статус_Позиции_Заявки_На_Закупку»

Имя атрибута	Тип данных	Обязательный	Описание	PK	FK
ID	INTEGER	Yes	Уникальный идентификатор статуса	Yes	No
Название	VARCHAR(50)	Yes	Наименование статуса	No	No

Приложение В

Спецификации физической схемы базы данных

1. Entity Relationship Diagram1



📁 1.1. author

1.1.1. Properties

Discriminator	N/A
Class model	N/A
Schema	public

1.1.2. Column Summary

Name	Description
 first_name	
d 	
 last_name	
ni  le_name	

1.1.3. Column Details

 first_name

Type	varchar
Length	100
Scale	0

id

Type	int4
Length	0
Scale	0
Primary Key	true

 last_name

Type	varchar
Length	100
Scale	0

 middle_name

Type	varchar
Length	100
Scale	0
Nullable	true

1.1.4. Indices

Name	Column
idx_author_last_name	last_name

1.1.5. Relationships

Relationship	From	To
 fkbook_autho683369	 author	 book_author

 1.2. book

1.2.1. Properties

Discriminator	N/A
Class model	N/A
Schema	public

1.2.2. Column Summary

Name	Description
 available_quantity	
 genreid	
<i>id</i> 	

Name	Description
 title	
 total_quantity	

1.2.3. Column Details

available_quantity

Type	int4
Length	0
Scale	0

genreid

Type	int4
Length	0
Scale	0
Foreign Key	true

id

Type	int4
Length	0
Scale	0
Primary Key	true

title

Type	varchar
Length	255
Scale	0

total_quantity

Type	int4
Length	0
Scale	0

1.2.4. Indices

Name	Column
idx_book_genreid	genreid
idx_book_title	title

1.2.5. Constraints

Name	Type	Constraint
book_title_key	Unique	UNIQUE(title)

1.2.6. Relationships

Relationship	From	To
fkBook_autho786924	 book	 book_author
fkBook_copy961686	 book	 book_copy
fkBook500794	 genre	 book
fkIssue_item694591	 book	 issue_item
fkManager_re167454	 book	 manager_request
fkPurchase_r396518	 book	 purchase_request_item
fkSupply_ite528158	 book	 supply_item

1.3. book_author

1.3.1. Properties

Discriminator	N/A
Class model	N/A
Schema	public

1.3.2. Column Summary

Name	Description
 authorid	

 bookid

1.3.3. Column Details

 *authorid*

Type	int4
Length	0
Scale	0
Primary Key	true
 Foreign Key	true

bookid

Type	int4
Length	0
Scale	0
Primary Key	true
Foreign Key	true

1.3.4. Indices

Name	Column
idx_book_author_authorid	authorid

1.3.5. Relationships

Relationship	From	To
fk_book_autho683369	 hor	 book_author
fk_book_autho786924	 book	 book_author

 1.4. book_copy

1.4.1. Properties

Discriminator	N/A
Class model	N/A
Schema	public

1.4.2. Column Summary

Name	Description
 arrival_date	
book_copy_statusid	
 bookid	
id 	
 inventory_number	

1.4.3. Column Details

 [arrival_date](#)

Type	date
Length	0
 Scale	0

[book_copy_statusid](#)

Type	int4
Length	0
Scale	0
Foreign Key	true

 [bookid](#)

Type	int4
Length	0
Scale	0
Foreign Key	true

 [id](#)

Type	int4
Length	0

Scale	0
Primary Key	true
 inventory_number	
Type	varchar
Length	50
Scale	0

1.4.4. Indices

Name	Column
idx_book_copy_bookid	bookid

idx_book_copy_statusid book_copy_statusid

1.4.5. Constraints

Name	Type	Constraint
book_copy_inventory_number_key	Unique	UNIQUE(inventory_number)

1.4.6. Relationships

Relationship	From	To
fkbook_copy587427	book_copy_status 	 book_copy
fkbook_copy961686	book 	 book_copy

 1.5. book_copy_status

1.5.1. Properties

Discriminator	N/A
Class model	N/A
Schema	public

1.5.2. Column Summary

Name	Description
 id	

name 

1.5.3. Column Details

id 	
Type	int4
Length	0
Scale	0
Primary Key	true
name 	
Type	varchar

Length	50
Scale	0

1.5.4. Constraints

Name	Type	Constraint
book_copy_status_name_key	Unique	UNIQUE(name)

1.5.5. Relationships

Relationship	From	To
fk_book_copy587427	book_copy 	book_copy 

 1.6. employee

1.6.1. Properties

Discriminator	N/A
Class model	N/A
Schema	public

1.6.2. Column Summary

Name	Description
 employment_date	
first  name	
id 	
last_name	
 libraryid	
middle  name	
 positionid	

1.6.3. Column Details

 employment_date

Type	date
Length	0
Scale	0

first name

Type	varchar
Length	100
Scale	0

id

Type	int4
------	------

Length	0
Scale	0
Primary Key	true

 *name*

Type	varchar
Length	100
Scale	0

 *libraryid*

Type	int4
Length	0
Scale	0
Foreign Key	true

 *middle_name*

Type	varchar
Length	100
Scale	0

 *positionid*

Type	int4
Length	0
Scale	0
Foreign Key	true

1.6.4. Indices

Name	Column
idx_employee_last_name	last_name
idx_employee_libraryid	libraryid
idx_employee_name_search	first_name, last_name
idx_employee_positionid	positionid

1.6.5. Relationships

Relationship	From	To
 fkemployee514608	 library	 employee
 fkemployee650198	 position	 employee
 fkissue18721	 employee	 issue
 fkmanager_re940456	 employee	 manager_request

 1.7. fine

1.7.1. Properties

Discriminator	N/A
Class model	N/A
Schema	public

1.7.2. Column Summary

Name	Description
 amount	
fine_reasonid	
 fine_statusid	
id	
 imposition_date	
 issuance_itemid	
 payment_date	
 read_receiptid	

1.7.3. Column Details

amount

Type	numeric
Length	10
Scale	2

fine_reasonid

Type	int4
Length	0
Scale	0
Foreign Key	true

fine_statusid

Type	int4
Length	0
Scale	0
Foreign Key	true

id

Type	int4
Length	0
Scale	0
Primary Key	true

imposition_date

Type	date
Length	0
Scale	0

 [issue_itemid](#)

Type	int4
Length	0
Scale	0

Foreign Key true

 [payment_date](#)

Type	date
Length	0
Scale	0

Nullable true

 [readerid](#)

Type	int4
Length	0
Scale	0

Foreign Key true

1.7.4. Indices

Name	Column
idx_fine_dates	imposition_date, payment_date
idx_fine_imposition_date	imposition_date
idx_fine_issue_itemid	issue_itemid
idx_fine_payment_date	payment_date
idx_fine_readerid	readerid
idx_fine_reasonid	fine_reasonid
idx_fine_statusid	fine_statusid

1.7.5. Relationships

Relationship	From	To
fkfine160352	 fine_status	 fine
fkfine244827	 fine_reason	 fine
fkfine6543	 issue_item	 fine
fkfine859344	 reader	 fine

 1.8. fine_reason

1.8.1. Properties

Discriminator	N/A
Class model	N/A
Schema	public

1.8.2. Column Summary

Name	Description
 id	

na

1.8.3. Column Details

<i>id</i> 	
Type	int4
Length	0
Scale	0
Primary Key	true

na

Type	varchar
Length	50
Scale	0

1.8.4. Constraints

Name	Type	Constraint
fine_reason_name_key	Unique	UNIQUE(name)

1.8.5. Relationships

Relationship	From	To
fkfine244827	<u>fine_reason</u> 	<u>fine</u> 

 1.9. fine_status

1.9.1. Properties

Discriminator	N/A
Class model	N/A
Schema	public

1.9.2. Column Summary

Name	Description
 id	

na

1.9.3. Column Details

id

Type	int4
Length	0
Scale	0
Primary Key	true

name

Type	varchar
Length	50
Scale	0

1.9.4. Constraints

Name	Type	Constraint
fine_status_name_key	Unique	UNIQUE(name)

1.9.5. Relationships

Relationship	From	To
fkfine160352	fine_status	fine

1.10. genre

1.10.1. Properties

Discriminator	N/A
Class model	N/A
Schema	public

1.10.2. Column Summary

Name	Description
id	

name

1.10.3. Column Details

id

Type	int4
Length	0
Scale	0
Primary Key	true

name

Type	varchar
Length	50
Scale	0

1.10.4. Constraints

Name	Type	Constraint
genre_name_key	Unique	UNIQUE(name)

1.10.5. Relationships

Relationship	From	To
fkBook500794	 genre	 book

 1.1. issue

1.11.1. Properties

Discriminator	N/A
Class model	N/A
Schema	public

1.11.2. Column Summary

Name	Description
 employeeid	

id

 issue_statusid
--

placed_return_date

 readerid
--

received_date

1.11.3. Column Details

 employeeid

Type	int4
Length	0
Scale	0
Foreign Key	true

 id

Type	int4
Length	0
Scale	0
Primary Key	true

 issue_statusid

Type	int4
------	------

Length 0

Scale	0
Foreign Key	true

 [planned_return_date](#)

Type	date
Length	0
Scale	0
Nullable	true

 [readerid](#)

Type	int4
Length	0
Scale	0
Foreign Key	true

 [request_date](#)

Type	date
Length	0
Scale	0

1.11.4. Indices

Name	Column
idx_issue_dates	planned_return_date, request_date
idx_issue_employeeid	employeeid
idx_issue_planned_return_date	planned_return_date
idx_issue_readerid	readerid
idx_issue_request_date	request_date
idx_issue_statusid	issue_statusid

1.11.5. Relationships

Relationship	From	To
 fkissue_item657321	 issue	 issue_item
 fkissue18721	 employee	 issue
 fkissue236550	 issue_status	 issue
 fkissue492432	 reader	 issue

 1.12. issue_item

1.12.1. Properties

Discriminator	N/A
Class model	N/A
Schema	public

1.12.2. Column Summary

Name	Description
 actual_return_date	
 bookid	
 id	
 issue_item_statusid	
 issueid	

1.12.3. Column Details

 actual_return_date

Type	date
Length	0
Scale	0
Nullable	true

 bookid

Type	int4
Length	0
Scale	0
Foreign Key	true

 id

Type	int4
Length	0
Scale	0
Primary Key	true

 issue_item_statusid

Type	int4
Length	0
Scale	0
Foreign Key	true

 issueid

Type	int4
Length	0
Scale	0
Foreign Key	true

1.12.4. Indices

Name	Column
idx_issue_item_actual_return_date	actual_return_date
idx_issue_item_bookid	bookid
idx_issue_item_issueid	issueid
idx_issue_item_statusid	issue_item_statusid

1.12.5. Relationships

Relationship	From	To
fk_issue_item6543	 issue_item	 fine
fk_issue_item139245	 issue_item_status	 issue_item
fk_issue_item657321	 issue	 issue_item
fk_issue_item694591	 book	 issue_item

 1.13. issue_item_status

1.13.1. Properties

Discriminator	N/A
Class model	N/A
Schema	public

1.13.2. Column Summary

Name	Description
 id	

name

1.13.3. Column Details

<i>id</i> 	
Type	int4
Length	0
Scale	0
Primary Key	true
<i>name</i> 	
Type	varchar
Length	50
Scale	0

1.13.4. Constraints

Name	Type	Constraint
------	------	------------

issue_item_status_name_key	Unique	UNIQUE(name)
----------------------------	--------	--------------

1.13.5. Relationships

Relationship	From	To
fkIssue_item139245	issue_item 	issue_item 

 1.14. issue_status

1.14.1. Properties

Discriminator	N/A
---------------	-----

Class model N/A

Schema	public
--------	--------

1.14.2. Column Summary

Name	Description
 id	

name 

1.14.3. Column Details

id 

Type	int4
------	------

Length 0

Scale	0
-------	---

Primary Key true

name 

Type	varchar
------	---------

Length 50

Scale	0
-------	---

1.14.4. Constraints

Name	Type	Constraint
issue_status_name_key	Unique	UNIQUE(name)

1.14.5. Relationships

Relationship	From	To
fkIssue236550	issue_status 	issue 

 1.15. library

1.15.1. Properties

Discriminator	N/A
---------------	-----

Class model N/A

Schema

public

1.15.2. Column Summary

Name	Description
 building_number	
id 	
 name	
ph 	
 street	

1.15.3. Column Details

building_number

Type	varchar
Length	10
Scale	0

id 

Type	int4
Length	0
Scale	0
Primary Key	true

name

Type	varchar
Length	255
Scale	0

phone

Type	varchar
Length	20
Scale	0

street

Type	varchar
Length	100
Scale	0

1.15.4. Indices

Name	Column
idx_library_phone	phone

1.15.5. Constraints

Name	Type	Constraint
library_name_key	Unique	UNIQUE(name)
library_phone_key	Unique	UNIQUE(phone)

1.15.6. Relationships

Relationship	From	To
fkEmployee514608	 library	 employee
fkReader5851	 library	 reader

 1.16. manager_request

1.16.1. Properties

Discriminator	N/A
Class model	N/A
Schema	public

1.16.2. Column Summary

Name	Description
 bookid	
creation_date	
 employeeid	
id 	
 manager_request_statusid	

1.16.3. Column Details

 bookid

Type	int4
Length	0
Scale	0
Foreign Key	true

 creation_date

Type	date
Length	0
 Scale	0

employeeid

Type	int4
Length	0
Scale	0

Foreign Key true

id 

Type	int4
------	------

Length	0
--------	---

Scale	0
-------	---

Primary Key true

 *manager_request_statusid*

Type	int4
------	------

Length	0
--------	---

Scale	0
-------	---

Foreign Key true

1.16.4. Indices

Name	Column
idx_manager_request_bookid	bookid
idx_manager_request_creation_date	creation_date
idx_manager_request_employeeid	employeeid
idx_manager_request_statusid	manager_request_statusid

1.16.5. Relationships

Relationship	From	To
fkmanager_re167454	 book	 manager_request
fkmanager_re173053	 manager_request_status	 manager_request
fkmanager_re940456	 employee	 manager_request

 1.17. manager_request_status

1.17.1. Properties

Discriminator	N/A
Class model	N/A
Schema	public

1.17.2. Column Summary

Name	Description
 id	

na 

1.17.3. Column Details

id 

Type	int4
Length	0
Scale	0
Primary Key	true

name

Type	varchar
Length	50
Scale	0

1.17.4. Constraints

Name	Type	Constraint
manager_request_status_name_key	Unique	UNIQUE(name)

1.17.5. Relationships

Relationship	From	To
fk_manager_re173053	manager_request_status	manager_request

1.18. position

1.18.1. Properties

Discriminator	N/A
Class model	N/A
Schema	public

1.18.2. Column Summary

Name	Description
id	

name

1.18.3. Column Details

id

Type	int4
Length	0
Scale	0
Primary Key	true

name

Type	varchar
Length	100
Scale	0

1.18.4. Constraints

Name	Type	Constraint
position_name_key	Unique	UNIQUE(name)

1.18.5. Relationships

Relationship	From	To
fk_employee650198	<u>position</u>	<u>employee</u>

1.19. purchase_request

1.19.1. Properties

Discriminator	N/A
Class model	N/A
Schema	public

1.19.2. Column Summary

Name	Description
creation_date	

id

purchase_request_statusid

supplierid

1.19.3. Column Details

creation_date

Type	date
Length	0
Scale	0

id

Type	int4
Length	0
Scale	0

Primary Key true

purchase_request_statusid

Type	int4
Length	0
Scale	0

Foreign Key true

supplierid

Type	int4
------	------

Length	0
Scale	0
Foreign Key	true

1.19.4. Indices

Name	Column
idx_purchase_request_creati on_date	creation_date
idx_purchase_request_status id	purchase_request_statusid
idx_purchase_request_suppli erid	supplierid

1.19.5. Relationships

Relationship	From	To
fkpurchase_r25238	 purchase_request	 purchase_request_item
fkpurchase_r536362	 purchase_request_status	 purchase_request
fkpurchase_r909394	 supplier	 purchase_request

 1.20. purchase_request_item

1.20.1. Properties

Discriminator	N/A
Class model	N/A
Schema	public

1.20.2. Column Summary

Name	Description
 bookid	
id 	
 purchase_request_ item_statusid	
 purchase_requestid	
 quantity	

1.20.3. Column Details

 [bookid](#)

Type	int4
Length	0
Scale	0
Foreign Key	true

id🔑

Type	int4
Length	0
Scale	0
Primary Key	true

purchase_request_item_statusid

Type	int4
Length	0
Scale	0
Foreign Key	true

purchase_requestid

Type	int4
Length	0
Scale	0
Foreign Key	true

quantity

Type	int4
Length	0
Scale	0

1.20.4. Indices

Name	Column
idx_purchase_request_item_	bookid bookid

idx_purchase_request_item_ purchase_requestid
requestid

idx_purchase_request_item_ purchase_request_item_statusid statusid

1.20.5. Relationships

Relationship	From	To
fkpurchase_r25238	 purchase_request	 purchase_request_item
fkpurchase_r396518	 book	 purchase_request_item
fkpurchase_r643340	 purchase_request_item_status	 purchase_request_item
fksupply_ite368767	 purchase_request_item	 supply_item

 1.21. purchase_request_item_status

1.21.1. Properties

Discriminator	N/A
---------------	-----

Class model	N/A
Schema	public

1.21.2. Column Summary

Name	Description
 id	

na 

1.21.3. Column Details

id 

Type	int4
Length	0
Scale	0
Primary Key	true

na 

Type	varchar
Length	50
Scale	0

1.21.4. Constraints

Name	Type	Constraint
purchase_request_item_status_name_key	Unique	UNIQUE(name)

1.21.5. Relationships

Relationship	From	To
 fkpurchase_r643340	 purchase_request_item_status	 purchase_request_item

1.22. purchase_request_status

1.22.1. Properties

Discriminator	N/A
Class model	N/A
Schema	public

1.22.2. Column Summary

Name	Description
 id	

na 

1.22.3. Column Details

id 

Type	int4
Length	0
Scale	0
Primary Key	true

 *name*

Type	varchar
Length	50
Scale	0

1.22.4. Constraints

Name	Type	Constraint
purchase_request_status_name_key	Unique	UNIQUE(name)

1.22.5. Relationships

Relationship	From	To
fk_purchase_r536362	<u>purchase_request_status</u>	<u>purchase_request</u>

 1.23. reader

1.23.1. Properties

Discriminator	N/A
Class model	N/A
Schema	public

1.23.2. Column Summary

Name	Description
 first_name	
 id	
 last_name	
 libraryid	
 middle_name	
 password	
 phone	
 reader_statusid	
 registration_date	

1.23.3. Column Details

 *first_name*

Type	varchar
------	---------

Length 100

Scale 0

id

Type int4

Length 0

Scale 0

Primary Key true

la_name

Type varchar

Length 100

Scale 0

lib_ryid

Type int4

Length 0

Scale 0

Foreign Key true

m_idle_name

Type varchar

Length 100

Scale 0

Nullable true

password

Type varchar

Length 255

Scale 0

pline

Type varchar

Length 20

Scale 0

rec_ier_statusid

Type int4

Length 0

Scale 0

Foreign Key true

registration_date

Type date

Length	0
Scale	0

1.23.4. Indices	
Name	Column
idx_reader_last_name	last_name
idx_reader_libraryid	libraryid
idx_reader_name_search	first_name, last_name
idx_reader_phone	phone
idx_reader_registration_date	registration_date
idx_reader_statusid	reader_statusid

1.23.5. Constraints		
Name	Type	Constraint
reader_phone_key	Unique	UNIQUE(phone)

1.23.6. Relationships		
Relationship	From	To
fkFine859344	 reader	 fine
fkIssue492432	 reader	 issue
fkReader264946	reader_status 	 reader
fkReader5851	 library	 reader

1.24. reader_status

1.24.1. Properties	
Discriminator	N/A
Class model	N/A
Schema	public

1.24.2. Column Summary	
Name	Description
 id	

na

1.24.3. Column Details

 id	
Type	int4
Length	0
Scale	0
Primary Key	true



Type	varchar
Length	50
Scale	0

1.24.4. Constraints

Name	Type	Constraint
reader_status_name_key	Unique	UNIQUE(name)

1.24.5. Relationships

Relationship	From	To
fk_reader264946	reader_status	reader

1.25. supplier

1.25.1. Properties

Discriminator	N/A
Class model	N/A
Schema	public

1.25.2. Column Summary

Name	Description
building_number	

city
id
name
phone
rating
street

1.25.3. Column Details

building_number

Type	varchar
Length	10
Scale	0

ci

Type	varchar
Length	50
Scale	0

id 

Type	int4
Length	0
Scale	0
Primary Key	true

name 

Type	varchar
Length	200
Scale	0

phone 

Type	varchar
Length	20
Scale	0

rating 

Type	int4
Length	0
Scale	0

street 

Type	varchar
Length	100
Scale	0

1.25.4. Indices

Name	Column
idx_supplier_name	name
idx_supplier_phone	phone

1.25.5. Constraints

Name	Type	Constraint
supplier_phone_key	Unique	UNIQUE(phone)

1.25.6. Relationships

Relationship	From	To
fk_purchase_r909394	 supplier	 purchase_request
fk_supply26053	 supplier	 supply

1.26. supply

1.26.1. Properties

Discriminator	N/A
---------------	-----

Class model	N/A
Schema	public

1.26.2. Column Summary

Name	Description
 delivery_date	
id	
 order_date	
 supplierid	
 supply_statusid	
total  cost	

1.26.3. Column Details

 delivery_date

Type	date
Length	0
Scale	0

id

Type	int4
Length	0
Scale	0
Primary Key	true

 order_date

Type	date
Length	0
Scale	0

 supplierid

Type	int4
Length	0
Scale	0
Foreign Key	true

 supply_statusid

Type	int4
Length	0
Scale	0
Foreign Key	true

to  l_cost

Type	numeric
Length	12
Scale	2

1.26.4. Indices

Name	Column
idx_supply_delivery_date	delivery_date
idx_supply_order_date	order_date
idx_supply_statusid	supply_statusid
idx_supply_supplierid	supplierid

1.26.5. Relationships

Relationship	From	To
fk_supply_ite394142	supply 	supply_ite 
fk_supply23249	supply_status 	supply 
fk_supply26053	supplier 	supply 

 1.27. supply_item

1.27.1. Properties

Discriminator	N/A
Class model	N/A
Schema	public

1.27.2. Column Summary

Name	Description
 bookid	
id	
 price_per_unit	
purchase_request_itemid	
 quantity	
 supply_item_status	
id	
 supplyid	

1.27.3. Column Details

 bookid

Type	int4
------	------

Length	0
Scale	0
Foreign Key	true

id 

Type	int4
Length	0
Scale	0
Primary Key	true

price_per_unit

Type	numeric
Length	10
Scale	2

purchase_request_itemid

Type	int4
Length	0
Scale	0
Foreign Key	true

quantity

Type	int4
Length	0
Scale	0

supply_item_statusid

Type	int4
Length	0
Scale	0
Foreign Key	true

supplyid

Type	int4
Length	0
Scale	0
Foreign Key	true

1.27.4. Indices

Name	Column
idx_supply_item_bookid	bookid
idx_supply_item_request_ite mid	purchase_request_itemid
idx_supply_item_statusid	supply_item_statusid

Name	Column
idx_supply_item_supplyid	supplyid

1.27.5. Relationships

Relationship	From	To
fksupply_ite368767	 purchase_request_item	 supply_item
fksupply_ite394142	 supply	 supply_item
fksupply_ite528158	 book	 supply_item
fksupply_ite653301	 supply_item_status	 supply_item

 1.28. supply_item_status

1.28.1. Properties

Discriminator	N/A
Class model	N/A
Schema	public

1.28.2. Column Summary

Name	Description
 id	

name 

1.28.3. Column Details

<i>id</i> 	
Type	int4
Length	0
Scale	0
Primary Key	true
<i>name</i> 	
Type	varchar
Length	50
Scale	0

1.28.4. Constraints

Name	Type	Constraint
supply_item_status_name_key	Unique	UNIQUE(name)

1.28.5. Relationships

Relationship	From	To
-fksupply_ite653301	 supply_item_status	 supply_item

 1.29. supply_status

1.29.1. Properties

Discriminator	N/A
Class model	N/A
Schema	public

1.29.2. Column Summary

Name	Description
 id	

name 

1.29.3. Column Details

id 

Type	int4
Length	0
Scale	0
Primary Key	true

name 

Type	varchar
Length	50
Scale	0

1.29.4. Constraints

Name	Type	Constraint
supply_status_name_key	Unique	UNIQUE(name)

1.29.5. Relationships

Relationship	From	To
 supply23249	 supply_status	 supply

Приложение Г

Скрипты (исходные коды) для создания объектов базы данных

-- Справочные таблицы

```
create table library (
  id          serial not null,
  name        varchar(255) not null unique,
  street       varchar(100) not null,
  building_number varchar(10) not null,
  phone        varchar(20) not null unique,
  primary key (id)
);
```

```
create table genre (
  id  serial not null,
  name varchar(50) not null unique,
  primary key (id)
);
```

```
create table position (
  id  serial not null,
  name varchar(100) not null unique,
  primary key (id)
);
```

```
create table author (
  id          serial not null,
  last_name   varchar(100) not null,
  first_name  varchar(100) not null,
  middle_name varchar(100),
  primary key (id)
);
```

```
create table supplier (
  id          serial not null,
  name        varchar(200) not null,
  city        varchar(50) not null,
  street       varchar(100) not null,
  building_number varchar(10) not null,
  phone        varchar(20) not null unique,
  rating       int4 default 3 not null,
  primary key (id)
);
```

-- Таблицы статусов

```
create table reader_status (
  id  serial not null,
  name varchar(50) not null unique,
  primary key (id)
);
```

```
create table issue_status (
  id  serial not null,
  name varchar(50) not null unique,
  primary key (id)
);
```

```
);
```

```
create table issue_item_status (  
  id serial not null,  
  name varchar(50) not null unique,  
  primary key (id)  
);
```

```
create table book_copy_status (  
  id serial not null,  
  name varchar(50) not null unique,  
  primary key (id)  
);
```

```
create table purchase_request_status (  
  id serial not null,  
  name varchar(50) not null unique,  
  primary key (id)  
);
```

```
create table purchase_request_item_status (  
  id serial not null,  
  name varchar(50) not null unique,  
  primary key (id)  
);
```

```
create table supply_status (  
  id serial not null,  
  name varchar(50) not null unique,  
  primary key (id)  
);
```

```
create table supply_item_status (  
  id serial not null,  
  name varchar(50) not null unique,  
  primary key (id)  
);
```

```
create table manager_request_status (  
  id serial not null,  
  name varchar(50) not null unique,  
  primary key (id)  
);
```

```
create table fine_status (  
  id serial not null,  
  name varchar(50) not null unique,  
  primary key (id)  
);
```

```
create table fine_reason (  
  id serial not null,
```

```

name varchar(50) not null unique,
primary key (id)
);

```

```
-- Основные таблицы
```

```

create table reader (
  id          serial not null,
  last_name   varchar(100) not null,
  first_name  varchar(100) not null,
  middle_name varchar(100),
  phone       varchar(20) not null unique,
  password    varchar(255) not null,
  registration_date date default CURRENT_DATE not null,
  reader_statusid int4 not null,
  libraryid   int4 not null,
  primary key (id)
);

```

```

create table book (
  id          serial not null,
  title       varchar(255) not null unique,
  total_quantity int4 not null check(total_quantity >= 0),
  available_quantity int4 not null check(available_quantity >= 0),
  genreid     int4 not null,
  primary key (id)
);

```

```

create table book_copy (
  id          serial not null,
  inventory_number varchar(50) not null unique,
  arrival_date date default CURRENT_DATE not null,
  bookid      int4 not null,
  book_copy_statusid int4 not null,
  primary key (id)
);

```

```

create table employee (
  id          serial not null,
  last_name   varchar(100) not null,
  first_name  varchar(100) not null,
  middle_name varchar(100) not null,
  employment_date date not null,
  positionid  int4 not null,
  libraryid   int4 not null,
  primary key (id)
);

```

```

create table issue (
  id          serial not null,
  request_date date default CURRENT_DATE not null,
  planned_return_date date,
  readerid    int4 not null,

```

```

issue_statusid int4 not null,
employeeid    int4 not null,
primary key (id)
);

```

```

create table issue_item (
  id          serial not null,
  actual_return_date date,
  issueid     int4 not null,
  issue_item_statusid int4 not null,
  bookid     int4 not null,
  primary key (id)
);

```

```

create table purchase_request (
  id          serial not null,
  creation_date date default CURRENT_DATE not null,
  supplierid  int4 not null,
  purchase_request_statusid int4 not null,
  primary key (id)
);

```

```

create table purchase_request_item (
  id          serial not null,
  quantity    int4 not null check(quantity > 0),
  purchase_request_item_statusid int4 not null,
  purchase_requestid int4 not null,
  bookid     int4 not null,
  primary key (id)
);

```

```

create table supply (
  id          serial not null,
  order_date  date default CURRENT_DATE not null,
  delivery_date date not null,
  total_cost  numeric(12, 2) not null,
  supplierid  int4 not null,
  supply_statusid int4 not null,
  primary key (id)
);

```

```

create table supply_item (
  id          serial not null,
  quantity    int4 not null check(quantity >= 0),
  price_per_unit numeric(10, 2) not null check(price_per_unit >= 0),
  supplyid    int4 not null,
  supply_item_statusid int4 not null,
  bookid     int4 not null,
  purchase_request_itemid int4 not null,
  primary key (id)
);

```

```

create table fine (
  id          serial not null,
  amount      numeric(10, 2) not null check(amount >= 0),
  imposition_date date default CURRENT_DATE not null,
  payment_date date,
  readerid    int4 not null,
  issue_itemid int4 not null,
  fine_statusid int4 not null,
  fine_reasonid int4 not null,
  primary key (id)
);

```

```

create table manager_request (
  id          serial not null,
  creation_date date default CURRENT_DATE not null,
  employeeid  int4 not null,
  manager_request_statusid int4 not null,
  bookid      int4 not null,
  primary key (id)
);

```

```

create table book_author (
  bookid  int4 not null,
  authorid int4 not null,
  primary key (bookid, authorid)
);

```

-- Внешние ключи с явными стратегиями ссылочной целостности

```

alter table reader
add constraint FKreader264946
foreign key (reader_statusid)
references reader_status (id)
on delete restrict
on update cascade;

```

```

alter table reader
add constraint FKreader5851
foreign key (libraryid)
references library (id)
on delete restrict
on update cascade;

```

```

alter table book
add constraint FKbook500794
foreign key (genreid)
references genre (id)
on delete restrict
on update cascade;

```

```

alter table book_copy
add constraint FKbook_copy961686
foreign key (bookid)

```

```
references book (id)
on delete restrict
on update cascade;
```

```
alter table book_copy
add constraint FKbook_copy587427
foreign key (book_copy_statusid)
references book_copy_status (id)
on delete restrict
on update cascade;
```

```
alter table employee
add constraint FKemployee650198
foreign key (positionid)
references position (id)
on delete restrict
on update cascade;
```

```
alter table employee
add constraint FKemployee514608
foreign key (libraryid)
references library (id)
on delete restrict
on update cascade;
```

```
alter table issue
add constraint FKissue492432
foreign key (readerid)
references reader (id)
on delete restrict
on update cascade;
```

```
alter table issue
add constraint FKissue236550
foreign key (issue_statusid)
references issue_status (id)
on delete restrict
on update cascade;
```

```
alter table issue
add constraint FKissue18721
foreign key (employeeid)
references employee (id)
on delete restrict
on update cascade;
```

```
alter table issue_item
add constraint FKissue_item657321
foreign key (issueid)
references issue (id)
on delete cascade
on update cascade;
```

```
alter table issue_item
add constraint FKissue_item139245
foreign key (issue_item_statusid)
references issue_item_status (id)
on delete restrict
on update cascade;
```

```
alter table issue_item
add constraint FKissue_item694591
foreign key (bookid)
references book (id)
on delete restrict
on update cascade;
```

```
alter table purchase_request
add constraint FKpurchase_r909394
foreign key (supplierid)
references supplier (id)
on delete restrict
on update cascade;
```

```
alter table purchase_request
add constraint FKpurchase_r536362
foreign key (purchase_request_statusid)
references purchase_request_status (id)
on delete restrict
on update cascade;
```

```
alter table purchase_request_item
add constraint FKpurchase_r643340
foreign key (purchase_request_item_statusid)
references purchase_request_item_status (id)
on delete restrict
on update cascade;
```

```
alter table purchase_request_item
add constraint FKpurchase_r25238
foreign key (purchase_requestid)
references purchase_request (id)
on delete cascade
on update cascade;
```

```
alter table purchase_request_item
add constraint FKpurchase_r396518
foreign key (bookid)
references book (id)
on delete restrict
on update cascade;
```

```
alter table supply
add constraint FKsupply26053
```

```
foreign key (supplierid)
references supplier (id)
on delete restrict
on update cascade;
```

```
alter table supply
add constraint FKsupply23249
foreign key (supply_statusid)
references supply_status (id)
on delete restrict
on update cascade;
```

```
alter table supply_item
add constraint FKsupply_ite394142
foreign key (supplyid)
references supply (id)
on delete cascade
on update cascade;
```

```
alter table supply_item
add constraint FKsupply_ite653301
foreign key (supply_item_statusid)
references supply_item_status (id)
on delete restrict
on update cascade;
```

```
alter table supply_item
add constraint FKsupply_ite528158
foreign key (bookid)
references book (id)
on delete restrict
on update cascade;
```

```
alter table supply_item
add constraint FKsupply_ite368767
foreign key (purchase_request_itemid)
references purchase_request_item (id)
on delete restrict
on update cascade;
```

```
alter table fine
add constraint FKfine859344
foreign key (readerid)
references reader (id)
on delete restrict
on update cascade;
```

```
alter table fine
add constraint FKfine6543
foreign key (issue_itemid)
references issue_item (id)
on delete restrict
```


on update cascade;

```
alter table fine
add constraint FKfine160352
foreign key (fine_statusid)
references fine_status (id)
on delete restrict
on update cascade;
```

```
alter table fine
add constraint FKfine244827
foreign key (fine_reasonid)
references fine_reason (id)
on delete restrict
on update cascade;
```

```
alter table manager_request
add constraint FKmanager_re940456
foreign key (employeeid)
references employee (id)
on delete restrict
on update cascade;
```

```
alter table manager_request
add constraint FKmanager_re173053
foreign key (manager_request_statusid)
references manager_request_status (id)
on delete restrict
on update cascade;
```

```
alter table manager_request
add constraint FKmanager_re167454
foreign key (bookid)
references book (id)
on delete restrict
on update cascade;
```

```
alter table book_author
add constraint FKbook_autho786924
foreign key (bookid)
references book (id)
on delete cascade
on update cascade;
```

```
alter table book_author
add constraint FKbook_autho683369
foreign key (authorid)
references author (id)
on delete cascade
on update cascade;
```

-- Индексы для внешних ключей

```

create index idx_reader_statusid on reader(reader_statusid);
create index idx_reader_libraryid on reader(libraryid);
create index idx_book_genreid on book(genreid);
create index idx_book_copy_bookid on book_copy(bookid);
create index idx_book_copy_statusid on book_copy(book_copy_statusid);
create index idx_employee_positionid on employee(positionid);
create index idx_employee_libraryid on employee(libraryid);
create index idx_issue_readerid on issue(readerid);
create index idx_issue_statusid on issue(issue_statusid);
create index idx_issue_employeeid on issue(employeeid);
create index idx_issue_item_issueid on issue_item(issueid);
create index idx_issue_item_statusid on issue_item(issue_item_statusid);
create index idx_issue_item_bookid on issue_item(bookid);
create index idx_purchase_request_supplierid on purchase_request(supplierid);
create index idx_purchase_request_statusid on purchase_request(purchase_request_statusid);
create index idx_purchase_request_item_statusid on
purchase_request_item(purchase_request_item_statusid);
create index idx_purchase_request_item_requestid on
purchase_request_item(purchase_requestid);
create index idx_purchase_request_item_bookid on purchase_request_item(bookid);
create index idx_supply_supplierid on supply(supplierid);
create index idx_supply_statusid on supply(supply_statusid);
create index idx_supply_item_supplyid on supply_item(supplyid);
create index idx_supply_item_statusid on supply_item(supply_item_statusid);
create index idx_supply_item_bookid on supply_item(bookid);
create index idx_supply_item_request_itemid on supply_item(purchase_request_itemid);
create index idx_fine_readerid on fine(readerid);
create index idx_fine_issue_itemid on fine(issue_itemid);
create index idx_fine_statusid on fine(fine_statusid);
create index idx_fine_reasonid on fine(fine_reasonid);
create index idx_manager_request_employeeid on manager_request(employeeid);
create index idx_manager_request_statusid on manager_request(manager_request_statusid);
create index idx_manager_request_bookid on manager_request(bookid);
create index idx_book_author_authorid on book_author(authorid);

```

-- Индексы для часто используемых полей

```

create index idx_reader_phone on reader(phone);
create index idx_reader_last_name on reader(last_name);
create index idx_reader_registration_date on reader(registration_date);
create index idx_book_title on book(title);
create index idx_author_last_name on author(last_name);
create index idx_employee_last_name on employee(last_name);
create index idx_issue_request_date on issue(request_date);
create index idx_issue_planned_return_date on issue(planned_return_date);
create index idx_issue_item_actual_return_date on issue_item(actual_return_date);
create index idx_purchase_request_creation_date on purchase_request(creation_date);
create index idx_supply_order_date on supply(order_date);
create index idx_supply_delivery_date on supply(delivery_date);
create index idx_fine_imposition_date on fine(imposition_date);
create index idx_fine_payment_date on fine(payment_date);
create index idx_manager_request_creation_date on manager_request(creation_date);
create index idx_supplier_phone on supplier(phone);

```

```
create index idx_supplier_name on supplier(name);  
create index idx_library_phone on library(phone);
```

-- Составные индексы для сложных запросов

```
create index idx_reader_name_search on reader(last_name, first_name);  
create index idx_employee_name_search on employee(last_name, first_name);  
create index idx_issue_dates on issue(request_date, planned_return_date);  
create index idx_fine_dates on fine(imposition_date, payment_date);
```

Приложение Д

Скрипты (исходные коды) для наполнения базы данных тестовыми данными

```
SET session_replication_role = 'replica';
```

```
-- =====
-- БЛОК 1: ТАБЛИЦА LIBRARY (БИБЛИОТЕКИ)
-- =====
INSERT INTO library (name, street, building_number, phone)
SELECT
    'Библиотека №' || i,
    CASE (i % 10)
        WHEN 0 THEN 'Ленина'
        WHEN 1 THEN 'Пушкина'
        WHEN 2 THEN 'Гагарина'
        WHEN 3 THEN 'Советская'
        WHEN 4 THEN 'Мира'
        WHEN 5 THEN 'Кирова'
        WHEN 6 THEN 'Чехова'
        WHEN 7 THEN 'Толстого'
        WHEN 8 THEN 'Горького'
        WHEN 9 THEN 'Центральная'
    END,
    (10 + (i % 90))::text,
    '+7910' || lpad((1000000 + i)::text, 7, '0')
FROM generate_series(1, 50) AS i;
```

```
-- =====
-- БЛОК 2: ТАБЛИЦА GENRE (ЖАНРЫ)
-- =====
INSERT INTO genre (name) VALUES
    ('Фантастика'), ('Детектив'), ('Роман'), ('Научная литература'),
    ('Историческая проза'), ('Приключения'), ('Поэзия'), ('Драма'),
    ('Детская литература'), ('Биография');
```

```
-- =====
-- БЛОК 3: ТАБЛИЦА POSITION (ДОЛЖНОСТИ)
-- =====
INSERT INTO position (name) VALUES
    ('Библиотекарь'),
    ('Работник книжного фонда'),
    ('Менеджер по поставкам');
```

```
-- =====
-- БЛОК 4: ТАБЛИЦА READER_STATUS (СТАТУСЫ ЧИТАТЕЛЯ)
-- =====
INSERT INTO reader_status (name) VALUES
    ('Активный'),
    ('Заблокирован');
```

```
-- =====
```

```
-- БЛОК 5: ТАБЛИЦА ISSUE_STATUS (СТАТУСЫ ВЫДАЧИ)
```

```
-- =====
```

```
INSERT INTO issue_status (name) VALUES
```

```
    ('Активна'),  
    ('Завершена'),  
    ('Просрочена'),  
    ('Отменена');
```

```
-- =====
```

```
-- БЛОК 6: ТАБЛИЦА ISSUE_ITEM_STATUS (СТАТУСЫ ПОЗИЦИЙ)
```

```
-- =====
```

```
INSERT INTO issue_item_status (name) VALUES
```

```
    ('Выдана'),  
    ('Возвращена');
```

```
-- =====
```

```
-- БЛОК 7: ТАБЛИЦА BOOK_COPY_STATUS (СТАТУСЫ ЭКЗЕМПЛЯРОВ)
```

```
-- =====
```

```
INSERT INTO book_copy_status (name) VALUES
```

```
    ('Доступна'),  
    ('Выдана'),  
    ('Списана');
```

```
-- =====
```

```
-- БЛОК 8: ТАБЛИЦА PURCHASE_REQUEST_STATUS (СТАТУСЫ ЗАЯВОК)
```

```
-- =====
```

```
INSERT INTO purchase_request_status (name) VALUES
```

```
    ('Новая'),  
    ('Одобрена'),  
    ('Выполнена'),  
    ('Отклонена');
```

```
-- =====
```

```
-- БЛОК 9: ТАБЛИЦА PURCHASE_REQUEST_ITEM_STATUS (СТАТУСЫ ПОЗ.)
```

```
-- =====
```

```
INSERT INTO purchase_request_item_status (name) VALUES
```

```
    ('Новая'),  
    ('Одобрена'),  
    ('Выполнена'),  
    ('Отклонена');
```

```
-- =====
```

```
-- БЛОК 10: ТАБЛИЦА SUPPLY_STATUS (СТАТУСЫ ПОСТАВОК)
```

```
-- =====
```

```
INSERT INTO supply_status (name) VALUES
```

```
    ('Ожидает'),  
    ('Получена');
```

('Проверена');

```
-- =====
-- БЛОК 11: ТАБЛИЦА SUPPLY_ITEM_STATUS (СТАТУСЫ ПОЗ. ПОСТАВОК)
-- =====
```

```
INSERT INTO supply_item_status (name) VALUES
  ('Получена'),
  ('Проверена');
```

```
-- =====
-- БЛОК 12: ТАБЛИЦА MANAGER_REQUEST_STATUS (СТАТУСЫ ЗАЯВОК МЕН.)
-- =====
```

```
INSERT INTO manager_request_status (name) VALUES
  ('Новая'),
  ('Отменена'),
  ('Выполнена');
```

```
-- =====
-- БЛОК 13: ТАБЛИЦА FINE_STATUS (СТАТУСЫ ШТРАФОВ)
-- =====
```

```
INSERT INTO fine_status (name) VALUES
  ('Активен'),
  ('Оплачен');
```

```
-- =====
-- БЛОК 14: ТАБЛИЦА FINE_REASON (ПРИЧИНЫ ШТРАФОВ)
-- =====
```

```
INSERT INTO fine_reason (name) VALUES
  ('Просрочка возврата'),
  ('Порча книги');
```

```
-- =====
-- БЛОК 15: ТАБЛИЦА SUPPLIER (ПОСТАВЩИКИ)
-- =====
```

```
INSERT INTO supplier (name, city, street, building_number, phone, rating)
SELECT
  'Поставщик ' || i,
  CASE (i % 5)
    WHEN 0 THEN 'Москва'
    WHEN 1 THEN 'Санкт-Петербург'
    WHEN 2 THEN 'Пенза'
    WHEN 3 THEN 'Самара'
    WHEN 4 THEN 'Казань'
  END,
  'Улица Поставщиков',
  (i % 100)::text,
  '+7800' || lpad((1000000 + i)::text, 7, '0'),
```

```
(random() * 2 + 3)::int
FROM generate_series(1, 100) AS i;
```

```
-- =====
-- БЛОК 16: ТАБЛИЦА AUTHOR (АВТОРЫ)
-- =====
```

```
INSERT INTO author (last_name, first_name, middle_name)
SELECT
  'Авторов' || (i % 100),
  CASE (i % 10) WHEN 0 THEN 'Александр' WHEN 1 THEN 'Алексей' WHEN 2 THEN
'Андрей' WHEN 3 THEN 'Дмитрий' WHEN 4 THEN 'Сергей' WHEN 5 THEN 'Иван' WHEN
6 THEN 'Михаил' WHEN 7 THEN 'Владимир' WHEN 8 THEN 'Николай' WHEN 9 THEN
'Евгений' END,
  CASE (i % 10) WHEN 0 THEN 'Александрович' WHEN 1 THEN 'Алексеевич' WHEN 2
THEN 'Андреевич' WHEN 3 THEN 'Дмитриевич' WHEN 4 THEN 'Сергеевич' WHEN 5
THEN 'Иванович' WHEN 6 THEN 'Михайлович' WHEN 7 THEN 'Владимирович' WHEN 8
THEN 'Николаевич' WHEN 9 THEN 'Евгеньевич' END
FROM generate_series(1, 20000) AS i;
```

```
-- =====
-- БЛОК 17: ТАБЛИЦА BOOK (КНИГИ)
-- =====
```

```
INSERT INTO book (title, total_quantity, available_quantity, genreid)
SELECT
  'Книга ' || i,
  (random() * 20 + 5)::int,
  (random() * 15 + 1)::int,
  (random() * 9 + 1)::int
FROM generate_series(1, 50000) AS i;
```

```
-- =====
-- БЛОК 18: ТАБЛИЦА EMPLOYEE (СОТРУДНИКИ)
-- =====
```

```
INSERT INTO employee (last_name, first_name, middle_name, employment_date, positionid,
libraryid)
SELECT
  'Сотрудников' || (i % 50),
  CASE (i % 10) WHEN 0 THEN 'Анна' WHEN 1 THEN 'Елена' WHEN 2 THEN 'Ольга'
WHEN 3 THEN 'Татьяна' WHEN 4 THEN 'Ирина' WHEN 5 THEN 'Наталья' WHEN 6 THEN
'Мария' WHEN 7 THEN 'Светлана' WHEN 8 THEN 'Юлия' WHEN 9 THEN 'Екатерина'
END,
  'Ивановна',
  CURRENT_DATE - (random() * 3650)::int,
  (random() * 2 + 1)::int,
  (random() * 49 + 1)::int
FROM generate_series(1, 500) AS i;
```

```
-- =====
```



```
-- БЛОК 19: ТАБЛИЦА READER (ЧИТАТЕЛИ)
```

```
-- =====
INSERT INTO reader (last_name, first_name, middle_name, phone, password, registration_date,
reader_statusid, libraryid)
SELECT
  'Читателей' || (i % 100),
  'Алексей', 'Петрович',
  '+7916' || lpad((1000000 + i)::text, 7, '0'),
  md5(random())::text,
  CURRENT_DATE - (random() * 365 * 3)::int,
  1,
  (random() * 49 + 1)::int
FROM generate_series(1, 10000) AS i;
```

```
-- =====
-- БЛОК 20: ТАБЛИЦА BOOK_COPY (ЭКЗЕМПЛЯРЫ КНИГ)
-- =====
INSERT INTO book_copy (inventory_number, arrival_date, bookid, book_copy_statusid)
SELECT
  'INV-' || lpad(i::text, 8, '0'),
  CURRENT_DATE - (random() * 365 * 5)::int,
  (random() * 49999 + 1)::int,
  CASE (random() * 100)::int WHEN 0 THEN 3 WHEN 1 THEN 2 ELSE 1 END
FROM generate_series(1, 150000) AS i;
```

```
-- =====
-- БЛОК 21: ТАБЛИЦА BOOK_AUTHOR (СВЯЗЬ КНИГА-АВТОР)
-- =====
INSERT INTO book_author (bookid, authorid)
SELECT
  (1 + floor(random() * 50000))::integer,
  (1 + floor(random() * 20000))::integer
FROM generate_series(1, 80000) ON CONFLICT DO NOTHING;
```

```
-- =====
-- БЛОК 22: ТАБЛИЦА ISSUE (ВЫДАЧИ)
-- =====
INSERT INTO issue (request_date, planned_return_date, readerid, issue_statusid, employeeid)
SELECT
  CURRENT_DATE - (random() * 365)::int,
  CURRENT_DATE - (random() * 300)::int + 14,
  (random() * 9999 + 1)::int,
  1,
  (random() * 499 + 1)::int
FROM generate_series(1, 20000) AS i;
```

```
-- =====
-- БЛОК 23: ТАБЛИЦА ISSUE_ITEM (ПОЗИЦИИ ВЫДАЧИ)
```

```
-- ВАЖНО: Реализация логики 80/20. Только 20% читателей нарушают.
-- =====
DO $$
DECLARE
    book_ids int[];
BEGIN
    SELECT ARRAY(SELECT DISTINCT bookid FROM book_copy) INTO book_ids;

    INSERT INTO issue_item (actual_return_date, issueid, issue_item_statusid, bookid)
    SELECT
        CASE
            WHEN (((SELECT readerid FROM issue WHERE id = sub.issue_id_gen) % 5) = 0)
            AND (random() > 0.3)
            THEN NULL -- "Плохой" читатель (20% базы)
            ELSE CURRENT_DATE - 1 -- "Хороший" читатель (80% базы)
        END,
        sub.issue_id_gen,
        CASE
            WHEN (((SELECT readerid FROM issue WHERE id = sub.issue_id_gen) % 5) = 0)
            AND (random() > 0.3)
            THEN 1
            ELSE 2
        END,
        book_ids[floor(random() * array_length(book_ids, 1) + 1)]
    FROM (
        SELECT (floor(random()*20000)+1)::int as issue_id_gen
        FROM generate_series(1, 40000)
    ) as sub;
END $$;
```

```
-- =====
-- БЛОК 24: ТАБЛИЦА FINE (ШТРАФЫ)
-- ВАЖНО: Штрафы назначаются только 20% читателей (ID кратно 5).
-- =====
INSERT INTO fine (amount, imposition_date, payment_date, readerid, issue_itemid,
fine_statusid, fine_reasonid)
SELECT
    (random() * 5000 + 100)::numeric(10,2),
    CURRENT_DATE - 10,
    NULL, -- Не оплачен
    i * 5, -- ID: 5, 10, 15... (Только 20% базы)
    (floor(random()*40000)+1)::int,
    1,
    1
FROM generate_series(1, 2000) AS i;
```

```
-- =====
-- БЛОК 25: ТАБЛИЦА PURCHASE_REQUEST (ЗАЯВКИ НА ЗАКУПКУ)
-- =====
INSERT INTO purchase_request (creation_date, supplierid, purchase_request_statusid)
```

```

SELECT
    CURRENT_DATE - (random() * 365 * 2)::int,
    (random() * 99 + 1)::int,
    (random() * 3 + 1)::int
FROM generate_series(1, 5000) AS i;

```

```

-- =====
-- БЛОК 26: ТАБЛИЦА PURCHASE_REQUEST_ITEM (ПОЗИЦИИ ЗАЯВОК)
-- =====

```

```

INSERT INTO purchase_request_item (quantity, purchase_request_item_statusid,
purchase_requestid, bookid)
SELECT
    (random() * 20 + 1)::int,
    (random() * 3 + 1)::int,
    (random() * 4999 + 1)::int,
    (random() * 49999 + 1)::int
FROM generate_series(1, 15000) AS i;

```

```

-- =====
-- БЛОК 27: ТАБЛИЦА SUPPLY (ПОСТАВКИ)
-- =====

```

```

INSERT INTO supply (order_date, delivery_date, total_cost, supplierid, supply_statusid)
SELECT
    CURRENT_DATE - (random() * 365 * 2)::int,
    CURRENT_DATE - (random() * 365 * 2 - 30)::int,
    (random() * 100000 + 1000)::numeric(12,2),
    (random() * 99 + 1)::int,
    CASE (random() * 100)::int WHEN 0 THEN 3 WHEN 1 THEN 1 ELSE 2 END
FROM generate_series(1, 3000) AS i;

```

```

-- =====
-- БЛОК 28: ТАБЛИЦА SUPPLY_ITEM (ПОЗИЦИИ ПОСТАВОК)
-- =====

```

```

INSERT INTO supply_item (quantity, price_per_unit, supplyid, supply_item_statusid, bookid,
purchase_request_itemid)
SELECT
    (random() * 50 + 1)::int,
    (random() * 1000 + 100)::numeric(10,2),
    (random() * 2999 + 1)::int,
    CASE (random() * 100)::int WHEN 0 THEN 1 ELSE 2 END,
    (random() * 49999 + 1)::int,
    (random() * 14999 + 1)::int
FROM generate_series(1, 10000) AS i;

```

```

-- =====
-- БЛОК 29: ТАБЛИЦА MANAGER_REQUEST (ЗАЯВКИ МЕНЕДЖЕРА)
-- =====

```

```

INSERT INTO manager_request (creation_date, employeeid, manager_request_statusid, bookid)

```

```

SELECT
  CURRENT_DATE - (random() * 180)::int,
  (random() * 499 + 1)::int,
  (random() * 2 + 1)::int,
  (random() * 49999 + 1)::int
FROM generate_series(1, 2000) AS i;

```

```

SET session_replication_role = 'origin';

```

```

-- ЗАБЕРШЕНИЕ

```

```

DO $$

```

```

BEGIN

```

```

  RAISE NOTICE 'Все 29 таблиц успешно заполнены!';
END $$;

```

```

-- 16. ПРОВЕРКА И СТАТИСТИКА

```

```

-----
-- Статистика по количеству записей в основных таблицах

```

```

DO $$

```

```

BEGIN

```

```

  RAISE NOTICE '=====';
  RAISE NOTICE 'СТАТИСТИКА ЗАПОЛНЕНИЯ БАЗЫ ДАННЫХ';
  RAISE NOTICE '=====';
  RAISE NOTICE 'Библиотеки: %', (SELECT COUNT(*) FROM library);
  RAISE NOTICE 'Читатели: %', (SELECT COUNT(*) FROM reader);
  RAISE NOTICE 'Книги (каталог): %', (SELECT COUNT(*) FROM book);
  RAISE NOTICE 'Авторы: %', (SELECT COUNT(*) FROM author);
  RAISE NOTICE 'Экземпляры книг: %', (SELECT COUNT(*) FROM book_copy);
  RAISE NOTICE 'Сотрудники: %', (SELECT COUNT(*) FROM employee);
  RAISE NOTICE 'Выдачи: %', (SELECT COUNT(*) FROM issue);
  RAISE NOTICE 'Позиции выдач: %', (SELECT COUNT(*) FROM issue_item);
  RAISE NOTICE 'Связи книги-авторы: %', (SELECT COUNT(*) FROM book_author);
  RAISE NOTICE 'Заявки на закупку: %', (SELECT COUNT(*) FROM purchase_request);
  RAISE NOTICE 'Позиции заявок: %', (SELECT COUNT(*) FROM
purchase_request_item);
  RAISE NOTICE 'Поставки: %', (SELECT COUNT(*) FROM supply);
  RAISE NOTICE 'Позиции поставок: %', (SELECT COUNT(*) FROM supply_item);
  RAISE NOTICE 'Штрафы: %', (SELECT COUNT(*) FROM fine);
  RAISE NOTICE 'Заявки менеджера: %', (SELECT COUNT(*) FROM manager_request);
  RAISE NOTICE '=====';
END $$;

```

```

-- Проверка целостности данных (простые проверки)

```

```

DO $$

```

```

BEGIN

```

```

  RAISE NOTICE '=====';
  RAISE NOTICE 'ПРОВЕРКА ЦЕЛОСТНОСТИ ДАННЫХ';
  RAISE NOTICE '=====';

```

```

-- Проверка читателей без библиотек

```

```

    IF EXISTS (SELECT 1 FROM reader r LEFT JOIN library l ON r.libraryid = l.id WHERE
l.id IS NULL) THEN
        RAISE NOTICE '✗ Найдены читатели без библиотек';
    ELSE
        RAISE NOTICE '✓ Все читатели имеют библиотеки';
    END IF;

    -- Проверка выданных книг без экземпляров
    IF EXISTS (SELECT 1 FROM issue_item ii LEFT JOIN book_copy bc ON ii.bookid =
bc.bookid WHERE bc.id IS NULL) THEN
        RAISE NOTICE '✗ Найдены выданные книги без экземпляров';
    ELSE
        RAISE NOTICE '✓ Все выданные книги имеют экземпляры';
    END IF;

    -- Проверка штрафов без читателей
    IF EXISTS (SELECT 1 FROM fine f LEFT JOIN reader r ON f.readerid = r.id WHERE r.id
IS NULL) THEN
        RAISE NOTICE '✗ Найдены штрафы без читателей';
    ELSE
        RAISE NOTICE '✓ Все штрафы имеют читателей';
    END IF;

    RAISE NOTICE '=====';
END $$;

```

Приложение Е

Исходные коды программных средств ведения данных

Appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "LibraryDb":
"Host=localhost;Database=city_libraries_db;Username=postgres;Password=1234"
  }
}
```

Program.cs

```
using CityLibrary.Models;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

builder.Services.AddControllers();
// Learn more about configuring Swagger/OpenAPI at
https://aka.ms/aspnetcore/swashbuckle
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddDbContext<LibraryDbContext>(options =>
    options.UseNpgsql(builder.Configuration.GetConnectionString("LibraryDb")));

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

app.UseAuthorization();

app.MapControllers();

app.Run();
```

SqlDtos.cs

```
namespace CityLibrary.DTO
{
    public class ReaderStatusCheck
    {
        public bool IsBlocked { get; set; }
        public bool HasOverdue { get; set; }
        public int ActiveIssuesCount { get; set; }
    }
}
```

```

public class ReturnInfo
{
    public int ItemId { get; set; }
    public int IssueId { get; set; }
    public int ReaderId { get; set; }
    public int BookId { get; set; }
    public DateOnly PlannedReturnDate { get; set; }
    public DateOnly? ActualReturnDate { get; set; }
}

public class GenreReportItem
{
    public string GenreName { get; set; }
    public long BorrowCount { get; set; }
}

public class OverdueReportItem
{
    public int Year { get; set; }
    public int Month { get; set; }
    public long DebtorsCount { get; set; }
}

public class ReaderInfoDTO
{
    public int Id { get; set; }
    public string FullName { get; set; }
    public string Phone { get; set; }
    public string StatusName { get; set; }
    public int ActiveIssues { get; set; }
    public int UnpaidFines { get; set; }
}

public class BookInfoDTO
{
    public int Id { get; set; }
    public string Title { get; set; }
    public string Genre { get; set; }
    public int AvailableCount { get; set; }
    public int TotalCount { get; set; }
}

}

```

LibrarianController.cs

```

using CityLibrary.Models;
using CityLibrary.DTO;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using System.Runtime.CompilerServices;

namespace CityLibrary.LibrarianApi.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class LibrarianController : ControllerBase
    {
        private readonly LibraryDbContext _context;
    }
}

```



```

public LibrarianController(LibraryDbContext context)
{
    _context = context;
}

//
=====
// ВСПОМОГАТЕЛЬНЫЕ МЕТОДЫ (Скрыты из Swagger)
// Реализуют Use Cases: "Прочитать данные читателя" и
"Проверить наличие книги"
//
=====

[HttpGet("reader/{id}")]
[ApiExplorerSettings(IgnoreApi = true)] // <--- Скрывает
метод из Swagger
public async Task<IActionResult> GetReaderInfo(int id)
{
    var query = @"
        SELECT
            r.id AS ""Id"",
            r.last_name || ' ' || r.first_name || ' ' ||
r.middle_name AS ""FullName"",
            r.phone AS ""Phone"",
            rs.name AS ""StatusName"",
            (SELECT COUNT(*) FROM issue WHERE readerid =
r.id AND issue_statusid = 1) AS ""ActiveIssues"",
            (SELECT COUNT(*) FROM fine WHERE readerid =
r.id AND fine_statusid = 1) AS ""UnpaidFines""
        FROM reader r
        JOIN reader_status rs ON r.reader_statusid =
rs.id

        WHERE r.id = {0}";

    var reader = await _context.Database
.SqlQuery<ReaderInfoDTO>(FormattableStringFactory.Create(query,
id))
        .FirstOrDefaultAsync();

    if (reader == null) return NotFound("Читатель не
найден");
    return Ok(reader);
}

[HttpGet("book/{id}")]
[ApiExplorerSettings(IgnoreApi = true)] // <--- Скрывает
метод из Swagger
public async Task<IActionResult> GetBookInfo(int id)
{

```

```

var query = @"
SELECT
    b.id AS ""Id"",
    b.title AS ""Title"",
    g.name AS ""Genre"",
    b.available_quantity AS ""AvailableCount"",
    b.total_quantity AS ""TotalCount""
FROM book b
JOIN genre g ON b.genreid = g.id
WHERE b.id = {0}";

var book = await _context.Database

.SqlQuery<BookInfoDTO>(FormattableStringFactory.Create(query,
id))

.FirstOrDefaultAsync();

if (book == null) return NotFound("Книга не
найденa");
return Ok(book);
}

//
=====
// ОСНОВНЫЕ ПРОЦЕССЫ (Видны в Swagger)
//
=====

// 1. Оформление выдачи книги (Процесс №1 + №6)
[HttpPost("issue-book")]
public async Task<IActionResult> IssueBook(int readerId,
int bookId, int librarianId)
{
    // ШАГ 1: Проверка лимитов (SQL)
    // Используем кавычки "", чтобы Postgres не понижал
регистр букв в названиях колонок
    var checkQuery = @"
SELECT
    (r.reader_statusid = 2) AS ""IsBlocked"",
    EXISTS (
        SELECT 1 FROM issue i_sub
        JOIN issue_item ii_sub ON i_sub.id =
ii_sub.issueid
        WHERE i_sub.readerid = r.id
        AND ii_sub.actual_return_date IS NULL
        AND i_sub.planned_return_date <
CURRENT_DATE
    ) AS ""HasOverdue"",
    (SELECT COUNT(*) FROM issue WHERE readerid =
r.id AND issue_statusid = 1) AS ""ActiveIssuesCount""
FROM reader r

```

```

        WHERE r.id = {0}";

        var status = await _context.Database

.SqlQuery<ReaderStatusCheck>(FormattableStringFactory.Create(che
ckQuery, readerId))
        .FirstOrDefaultAsync();

        if (status == null) return NotFound("Читатель не
найден");
        if (status.IsBlocked) return BadRequest("ОТКАЗ:
Читатель заблокирован.");
        if (status.HasOverdue) return BadRequest("ОТКАЗ: У
читателя есть просроченные книги.");
        if (status.ActiveIssuesCount >= 3) return
BadRequest("ОТКАЗ: Превышен лимит активных выдач (макс. 3).");

        // ШАГ 2: Выполнение транзакции выдачи
        using var transaction = await
_context.Database.BeginTransactionAsync();
        try
        {
            var requestDate =
DateOnly.FromDateTime(DateTime.Now);
            var plannedDate =
DateOnly.FromDateTime(DateTime.Now.AddDays(14));

            // Вставка в issue и получение ID
            var issueIds = _context.Database
                .SqlQuery<int>($"@
                INSERT INTO issue (readerid, employeeid,
request_date, planned_return_date, issue_statusid)
                VALUES ({readerId}, {librarianId},
{requestDate}, {plannedDate}, 1)
                RETURNING id")
                .AsEnumerable(); // <--- ЭТО
РЕШАЕТ ОШИБКУ "Non-composable SQL"

            var issueId = issueIds.Single();
            // Вставка позиции выдачи
            await _context.Database.ExecuteSqlAsync($"@
                INSERT INTO issue_item (issueid, bookid,
issue_item_statusid)
                VALUES ({issueId}, {bookId}, 1)");

            // Обновление количества книг
            var rowsAffected = await
_context.Database.ExecuteSqlAsync($"@
                UPDATE book
                SET available_quantity = available_quantity
- 1
                WHERE id = {bookId} AND available_quantity >
0");

```

```

        if (rowsAffected == 0)
        {
            await transaction.RollbackAsync();
            return BadRequest("ОТКАЗ: Книги нет в
наличии.");
        }

        await transaction.CommitAsync();
        return Ok(new { Message = "Книга успешно
выдана", IssueId = issueId });
    }
    catch (Exception ex)
    {
        await transaction.RollbackAsync();
        return StatusCode(500, $"Ошибка БД:
{ex.Message}");
    }
}

// 2. Оформление возврата книги (Процесс №2 + №3 + №7)
[HttpPost("return-book")]
public async Task<IActionResult> ReturnBook(int
issueItemId, bool isDamaged)
{
    // Получаем информацию о выдаче
    var infoQuery = @"
        SELECT
            ii.id AS ""ItemId"",
            ii.issueid AS ""IssueId"",
            ii.bookid AS ""BookId"",
            i.readerid AS ""ReaderId"",
            i.planned_return_date AS
""PlannedReturnDate"",
            ii.actual_return_date AS
""ActualReturnDate""
        FROM issue_item ii
        JOIN issue i ON ii.issueid = i.id
        WHERE ii.id = {0}";

    var info = await _context.Database
.SqlQuery<ReturnInfo>(FormattableStringFactory.Create(infoQuery,
issueItemId))
        .FirstOrDefaultAsync();

    if (info == null) return NotFound("Запись о выдаче
не найдена");
    if (info.ActualReturnDate != null) return
BadRequest("Эта книга уже возвращена.");

    using var transaction = await
_context.Database.BeginTransactionAsync();

```

```

try
{
    var today = DateOnly.FromDateTime(DateTime.Now);

    // Фиксация возврата
    await _context.Database.ExecuteSqlAsync($"@\"
        UPDATE issue_item
        SET actual_return_date = {today},
            issue_item_statusid = 2
        WHERE id = {issueItemId}\"");

    // Автообновление статуса книги (на полку или
    списание)
    if (!isDamaged)
    {
        await _context.Database.ExecuteSqlAsync($"@\"
            UPDATE book
            SET          available_quantity          =
available_quantity + 1
            WHERE id = {info.BookId}\"");
    }
    else
    {
        // Штраф за порчу (1500 руб, причина 2)
        await _context.Database.ExecuteSqlAsync($"@\"
            INSERT INTO fine (readerid,
issue_itemid,      fine_reasonid,      fine_statusid,      amount,
imposition_date)
            VALUES ({info.ReaderId}, {issueItemId},
2, 1, 1500.00, {today})\"");
    }

    // Штраф за просрочку (автоматически)
    if (info.PlannedReturnDate < today)
    {
        await _context.Database.ExecuteSqlAsync($"@\"
            INSERT INTO fine (readerid,
issue_itemid,      fine_reasonid,      fine_statusid,      amount,
imposition_date)
            VALUES ({info.ReaderId}, {issueItemId},
1, 1, 100.00, {today})\"");
    }

    // Закрытие выдачи (если все книги возвращены)
    var pendingCount = await _context.Database
        .SqlQuery<int>($"SELECT COUNT(*) AS
\"Value\" FROM issue_item WHERE issueid = {info.IssueId} AND
actual_return_date IS NULL")
        .SingleAsync();

    if (pendingCount == 0)
    {
        await _context.Database.ExecuteSqlAsync($"@\"

```

```

        UPDATE issue
        SET issue_statusid = 2
        WHERE id = {info.IssueId}");
    }

    await transaction.CommitAsync();
    return Ok(new { Message = isDamaged ? "Книга
принята с пометкой 'Повреждена'." : "Книга успешно возвращена."
});
}
catch (Exception ex)
{
    await transaction.RollbackAsync();
    return StatusCode(500, $"Ошибка БД:
{ex.Message}");
}
}

// 3. Отчет по популярным жанрам (Процесс №4)
[HttpGet("popular-genres")]
public async Task<IActionResult> GetPopularGenres()
{
    var query = @"
        SELECT
            g.name AS ""GenreName"",
            COUNT(ii.id) AS ""BorrowCount""
        FROM issue_item ii
        JOIN book b ON ii.bookid = b.id
        JOIN genre g ON b.genreid = g.id
        GROUP BY g.name
        ORDER BY ""BorrowCount"" DESC
        LIMIT 10";

    var report = await _context.Database
.SqlQuery<GenreReportItem>(FormattableStringFactory.Create(query
))
.ToListAsync();

    return Ok(report);
}

// 4. Отчет по должникам (Процесс №5)
[HttpGet("overdue-stats")]
public async Task<IActionResult> GetOverdueStats()
{
    var query = @"
        SELECT
            CAST(EXTRACT(YEAR
i.planned_return_date) AS INTEGER) AS ""Year"",
            CAST(EXTRACT(MONTH
i.planned_return_date) AS INTEGER) AS ""Month"",
            FROM
            FROM
    
```

```

COUNT(DISTINCT      i.readerid)      AS
""DebtorsCount""
    FROM issue i
    JOIN issue_item ii ON i.id = ii.issueid
    WHERE ii.actual_return_date IS NULL
      AND i.planned_return_date < CURRENT_DATE
    GROUP BY ""Year"", ""Month""
    ORDER BY ""Year"", ""Month"";

    var stats = await _context.Database

.SqlQuery<OverdueReportItem>(FormattableStringFactory.Create(query))
    .ToListAsync();

    return Ok(stats);
}
}
}

```